

Motor Insurance SQL Assignment – Table Design

Business Model:

The business requirement is a motor insurance platform where users create quotes, calculate premium, receive payment, and issue policies. The same platform supports both B2B (broker-driven) and B2C (direct) sales.

Schema Design:

- Role-based operations (admin, underwriter, operational, broker, sales agent)
- Quote-to-policy operations
- Broker operations
- Regional reporting
- Vehicle, products, customers databases
- Payment completion (In different countries)

Tables:

- regions, products, premium_rates
- app_users, brokers, customers, vehicles
- quotes, payments, policies

Table Definitions:

1. regions

Purpose:

Stores business location details for reporting

Columns and Constraints:

- region_id INT PRIMARY KEY: every region must have one stable unique identifier
- region_name/state_name/city_name NOT NULL
- status ENUM('ACTIVE','INACTIVE') DEFAULT 'ACTIVE': region can be deactivated without deleting historical data

2. app_users

Purpose:

Stores all users who log in and act in the flow

Columns and Constraints:

- user_id PRIMARY KEY: Stable user identity
- role ENUM(...) NOT NULL
- email UNIQUE NOT NULL: login/contact identifier cannot duplicate
- region_id FK -> regions(region_id)
- manager_user_id FK -> app_users(user_id): Hierarchy (for broker/sales reporting chain)

- status ENUM('ACTIVE','BLOCKED'): Supports user blocking from admin controls
- created_at DEFAULT CURRENT_TIMESTAMP: Audit timestamp

3. brokers

Purpose:

Stores broker organizations and financial configuration

Columns and Constraints:

- broker_id PRIMARY KEY
- broker_code UNIQUE: practical external/business identifier
- region_id FK -> regions: broker belongs to a location
- linked_user_id FK -> app_users: User linkage for organization mapping
- prepaid_balance DECIMAL(12,2): required because broker debit/prepaid model is in requirements
- commission_pct DECIMAL(5,2): required for commission payout
- status ENUM('ACTIVE','BLOCKED'): operational control

4. customers

Purpose:

Stores insured customer identity and contact profile

Columns and Constraints:

- customer_id PRIMARY KEY
- email UNIQUE NOT NULL: avoid duplicate customer records by email
- national_id UNIQUE NOT NULL: KYC identity
- dob/gender/phone NOT NULL: Mandatory profile details
- created_at

5. vehicles

Purpose:

Stores insurable assets linked to customers

Columns and Constraints:

- vehicle_id PRIMARY KEY
- customer_id FK -> customers: vehicle must belong to a known customer
- registration_no UNIQUE NOT NULL
- vehicle_category ENUM('PRIVATE','COMMERCIAL','TAXI','MOTORCYCLE','SPORTS')
- sum_insured DECIMAL(12,2): Premium input
- status ENUM for deactivation without hard delete

6. products

Purpose:

Stores insurance plan definitions

Columns and Constraints:

- product_id PRIMARY KEY
- cover_type ENUM('COMPREHENSIVE','TPL'): Approved cover models
- has_roadside_assist / has_zero_dep as feature flags
- base_tax_pct DECIMAL(5,2):
- currency_code CHAR(3) default INR
- status ENUM for product activation and control

7. premium_rates

Purpose:

Stores pricing rules by product, category, risk band, and validity period

Columns and Constraints:

- rate_id PRIMARY KEY
- product_id FK -> products: Rates tied to valid products only
- risk_band ENUM('LOW','MEDIUM','HIGH'): Risk classification
- base_rate_pct / addon_rate_pct / min_premium
- valid_from and valid_to:
- status ENUM for rate_id management

8. quotes

Purpose:

Transactional table for quote generation and premium output

Columns and Constraints:

- quote_id PRIMARY KEY and quote_no UNIQUE
- customer_id FK, vehicle_id FK, product_id FK, rate_id FK
- source_channel ENUM('B2B','B2C')
- broker_id nullable FK: null allowed because B2C direct quotes may not use broker
- sales_user_id FK: Tracks which user created quote
- addon_selected flag: Optional cover selection
- premium_before_tax, tax_amount, total_premium DECIMAL
- quote_status ENUM('DRAFT','REFERRED','APPROVED','PAID','EXPIRED')

9. payments

Purpose:

Stores payment attempt and settlement details for quote

Columns and Constraints:

- payment_id PRIMARY KEY
- quote_id UNIQUE FK -> quotes: one payment record per quote in this assignment model
- payment_method ENUM includes BROKER_DEBIT
- paid_currency + exchange_rate_to_inr + paid_amount_inr: required for foreign currency use case
- payment_status ENUM('PENDING','SUCCESS','FAILED')
- paid_at nullable: pending payments may not have timestamp

10. policies

Purpose:

Stores final issued policy contract and post-issue status

Columns and Constraints:

- policy_id PRIMARY KEY and policy_no UNIQUE
- quote_id UNIQUE FK: each quote can map to at most one policy
- payment_id UNIQUE nullable FK
- approval_user_id FK -> app_users: captures operational/underwriter approval authority
- policy_status ENUM('PENDING','ACTIVE','CANCELLED','LAPSED')
- debit_note_no and credit_note_no included to align with requirement for note generation

Relationship map and process flow:

- one region -> many app_users
- one region -> many brokers
- one customer -> many vehicles
- one product -> many premium_rates
- one customer/vehicle/product/rate combination -> one quote record
- one quote -> one payment record
- one quote -> one policy record

Business process sequence:

- user logs in (app_users)
- quote created (quotes) using customer + vehicle + product + premium_rate
- payment captured (payments), including foreign currency conversion when needed
- policy issued/updated (policies), with approver and status

Database Creation:

The screenshot shows the SQL Developer interface. The Navigator pane on the left displays the SCHEMAS tree with folders for college, company, Tables, Views, Stored Procedures, Functions, and dmart. The main query window shows the following SQL code:

```
1
2
3 CREATE DATABASE motor_insurance_assignment;
4 USE motor_insurance_assignment;
5
6
7
8
9
10
```

The Output pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec

Table Creation:

Regions

The screenshot shows the SQL Developer interface. The main query window shows the following SQL code:

```
19 --
20
21 CREATE TABLE regions (
22     region_id INT PRIMARY KEY,
23     region_name VARCHAR(40) NOT NULL,
24     state_name VARCHAR(60) NOT NULL,
25     city_name VARCHAR(60) NOT NULL,
26     status ENUM('ACTIVE', 'INACTIVE') NOT NULL DEFAULT 'ACTIVE'
27 )
28
```

The Output pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec
3	03:02:52	CREATE TABLE regions (region_id INT PRIMARY KEY, region_name VARCHAR(40) NOT NULL, state...	0 row(s) affected	0.109 sec

App_users

The screenshot shows the SQL Developer interface. The main query window shows the following SQL code:

```
13
14
15 CREATE TABLE app_users (
16     user_id INT PRIMARY KEY,
17     full_name VARCHAR(80) NOT NULL,
18     role ENUM('ADMIN', 'UNDERWRITER', 'OPERATIONAL', 'BROKER', 'SALES_AGENT') NOT NULL,
19     email VARCHAR(120) NOT NULL UNIQUE,
20     phone VARCHAR(20) NOT NULL,
21     region_id INT NOT NULL,
22     manager_user_id INT NOT NULL,
23     status ENUM('ACTIVE', 'BLOCKED') NOT NULL DEFAULT 'ACTIVE',
24     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
25     CONSTRAINT fk_users_region FOREIGN KEY (region_id) REFERENCES regions(region_id),
26     CONSTRAINT fk_users_manager FOREIGN KEY (manager_user_id) REFERENCES app_users(user_id)
27 );
28
```

The Output pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec
3	03:02:52	CREATE TABLE regions (region_id INT PRIMARY KEY, region_name VARCHAR(40) NOT NULL, state...	0 row(s) affected	0.109 sec
4	03:06:09	CREATE TABLE app_users (user_id INT PRIMARY KEY, full_name VARCHAR(80) NOT NULL, role E...	0 row(s) affected	0.062 sec

Brokers :

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane shows a tree view with 'college', 'company', 'Tables', 'Views', 'Stored Procedures', 'Functions', and 'dmar'. The 'Information' pane shows 'No object selected'. The main pane displays the SQL script for creating the 'brokers' table:

```
28
29 CREATE TABLE brokers (
30     broker_id INT PRIMARY KEY,
31     broker_code VARCHAR(20) NOT NULL UNIQUE,
32     broker_org_name VARCHAR(120) NOT NULL,
33     contact_name VARCHAR(80) NOT NULL,
34     region_id INT NOT NULL,
35     linked_user_id INT NOT NULL,
36     prepaid_balance DECIMAL(12,2) NOT NULL,
37     commission_pct DECIMAL(5,2) NOT NULL,
38     status ENUM('ACTIVE', 'BLOCKED') NOT NULL DEFAULT 'ACTIVE',
39     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
40     CONSTRAINT fk_brokers_region FOREIGN KEY (region_id) REFERENCES regions(region_id),
41     CONSTRAINT fk_brokers_user FOREIGN KEY (linked_user_id) REFERENCES app_users(user_id)
42 );
```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec
3	03:02:52	CREATE TABLE regions (region_id INT PRIMARY KEY, region_name VARCHAR(40) NOT NULL, state...	0 row(s) affected	0.109 sec
4	03:06:09	CREATE TABLE app_users (user_id INT PRIMARY KEY, full_name VARCHAR(80) NOT NULL, role E...	0 row(s) affected	0.062 sec
5	03:06:49	CREATE TABLE brokers (broker_id INT PRIMARY KEY, broker_code VARCHAR(20) NOT NULL UNIQ...	0 row(s) affected	0.063 sec

Customers:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane shows a tree view with 'college', 'company', 'Tables', 'Views', 'Stored Procedures', 'Functions', and 'dmar'. The 'Information' pane shows 'No object selected'. The main pane displays the SQL script for creating the 'customers' table:

```
43
44 CREATE TABLE customers (
45     customer_id INT PRIMARY KEY,
46     first_name VARCHAR(50) NOT NULL,
47     last_name VARCHAR(50) NOT NULL,
48     dob DATE NOT NULL,
49     gender ENUM('M', 'F', 'O') NOT NULL,
50     email VARCHAR(120) NOT NULL UNIQUE,
51     phone VARCHAR(20) NOT NULL,
52     national_id VARCHAR(30) NOT NULL UNIQUE,
53     city_name VARCHAR(40) NOT NULL,
54     state_name VARCHAR(40) NOT NULL,
55     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
56 );
```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec
3	03:02:52	CREATE TABLE regions (region_id INT PRIMARY KEY, region_name VARCHAR(40) NOT NULL, state...	0 row(s) affected	0.109 sec
4	03:06:09	CREATE TABLE app_users (user_id INT PRIMARY KEY, full_name VARCHAR(80) NOT NULL, role E...	0 row(s) affected	0.062 sec
5	03:06:49	CREATE TABLE brokers (broker_id INT PRIMARY KEY, broker_code VARCHAR(20) NOT NULL UNIQ...	0 row(s) affected	0.063 sec
6	03:07:50	CREATE TABLE customers (customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, la...	0 row(s) affected	0.031 sec

Vehicles:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SCHEMAS' pane shows a tree view with 'college', 'company', 'Tables', 'Views', 'Stored Procedures', 'Functions', and 'dmar'. The 'Information' pane shows 'No object selected'. The main pane displays the SQL script for creating the 'vehicles' table:

```
58
59 CREATE TABLE vehicles (
60     vehicle_id INT PRIMARY KEY,
61     customer_id INT NOT NULL,
62     make VARCHAR(50) NOT NULL,
63     model VARCHAR(40) NOT NULL,
64     model_year YEAR NOT NULL,
65     body_type VARCHAR(40) NOT NULL,
66     vehicle_category ENUM('PRIVATE', 'COMMERCIAL', 'TAXI', 'MOTORCYCLE', 'SPORTS') NOT NULL,
67     registration_no VARCHAR(20) NOT NULL UNIQUE,
68     sum_insured DECIMAL(12,2) NOT NULL,
69     status ENUM('ACTIVE', 'INACTIVE') NOT NULL DEFAULT 'ACTIVE',
70     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
71     CONSTRAINT fk_vehicles_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
72 );
```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec
3	03:02:52	CREATE TABLE regions (region_id INT PRIMARY KEY, region_name VARCHAR(40) NOT NULL, state...	0 row(s) affected	0.109 sec
4	03:06:09	CREATE TABLE app_users (user_id INT PRIMARY KEY, full_name VARCHAR(80) NOT NULL, role E...	0 row(s) affected	0.062 sec
5	03:06:49	CREATE TABLE brokers (broker_id INT PRIMARY KEY, broker_code VARCHAR(20) NOT NULL UNIQ...	0 row(s) affected	0.063 sec
6	03:07:50	CREATE TABLE customers (customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, la...	0 row(s) affected	0.031 sec
7	03:09:05	CREATE TABLE vehicles (vehicle_id INT PRIMARY KEY, customer_id INT NOT NULL, make VARCH...	0 row(s) affected	0.047 sec

Products:

The screenshot shows a SQL IDE interface with a query editor, a schema navigator, and an output window. The query editor contains the following SQL code:

```
73 CREATE TABLE products (  
74     product_id INT PRIMARY KEY,  
75     product_name VARCHAR(100) NOT NULL,  
76     cover_type ENUM('COMPREHENSIVE', 'TPL') NOT NULL,  
77     has_roadside_assist INT NOT NULL DEFAULT 0,  
78     has_zero_dep INT NOT NULL DEFAULT 0,  
79     base_tax_pct DECIMAL(5,2) NOT NULL,  
80     currency_code CHAR(3) NOT NULL DEFAULT 'INR',  
81     status ENUM('ACTIVE', 'INACTIVE') NOT NULL DEFAULT 'ACTIVE'  
82 );  
83 DROP TABLE products;
```

The output window shows the execution results of the query. The first two rows show the creation of the database and the use of the database. The subsequent rows show the creation of various tables and the dropping of the 'products' table. The final row shows the creation of the 'products' table.

#	Time	Action	Message	Duration / Fetch
1	02:59:51	CREATE DATABASE motor_insurance_assignment	1 row(s) affected	0.047 sec
2	02:59:51	USE motor_insurance_assignment	0 row(s) affected	0.015 sec
3	03:02:52	CREATE TABLE regions (region_id INT PRIMARY KEY, region_name VARCHAR(40) NOT NULL, state...	0 row(s) affected	0.109 sec
4	03:06:09	CREATE TABLE app_users (user_id INT PRIMARY KEY, full_name VARCHAR(80) NOT NULL, role E...	0 row(s) affected	0.062 sec
5	03:06:49	CREATE TABLE brokers (broker_id INT PRIMARY KEY, broker_code VARCHAR(20) NOT NULL UNIQ...	0 row(s) affected	0.063 sec
6	03:07:50	CREATE TABLE customers (customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, la...	0 row(s) affected	0.031 sec
7	03:09:05	CREATE TABLE vehicles (vehicle_id INT PRIMARY KEY, customer_id INT NOT NULL, make VARCH...	0 row(s) affected	0.047 sec
8	03:09:52	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	0 row(s) affected. 2 warning(s): 1681 Integer display width is deprecated and will be removed in a future release...	0.032 sec
9	03:10:28	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	Error Code: 1050 Table 'products' already exists	0.032 sec
10	03:10:49	DROP TABLE products	0 row(s) affected	0.046 sec
11	03:10:55	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	0 row(s) affected. 1 warning(s): 1681 Integer display width is deprecated and will be removed in a future release...	0.031 sec
12	03:11:25	DROP TABLE products	0 row(s) affected	0.032 sec
13	03:11:29	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	0 row(s) affected	0.031 sec

Premium_Rates:

The screenshot shows a SQL IDE interface with a query editor, a schema navigator, and an output window. The query editor contains the following SQL code:

```
85 CREATE TABLE premium_rates (  
86     rate_id INT PRIMARY KEY,  
87     product_id INT NOT NULL,  
88     vehicle_category ENUM('PRIVATE', 'COMMERCIAL', 'TAXI', 'MOTORCYCLE', 'SPORTS') NOT NULL,  
89     risk_band ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,  
90     base_rate_pct DECIMAL(5,2) NOT NULL,  
91     add_on_rate_pct DECIMAL(5,2) NOT NULL,  
92     min_premium DECIMAL(10,2) NOT NULL,  
93     valid_from DATE NOT NULL,  
94     valid_to DATE NOT NULL,  
95     status ENUM('ACTIVE', 'INACTIVE') NOT NULL DEFAULT 'ACTIVE',  
96     CONSTRAINT fk_rates_product FOREIGN KEY (product_id) REFERENCES products(product_id)  
97 );
```

The output window shows the execution results of the query. The first two rows show the creation of the database and the use of the database. The subsequent rows show the creation of various tables and the dropping of the 'products' table. The final row shows the creation of the 'premium_rates' table.

#	Time	Action	Message	Duration / Fetch
4	03:06:09	CREATE TABLE app_users (user_id INT PRIMARY KEY, full_name VARCHAR(80) NOT NULL, role...	0 row(s) affected	0.062 sec
5	03:06:49	CREATE TABLE brokers (broker_id INT PRIMARY KEY, broker_code VARCHAR(20) NOT NULL UNIQ...	0 row(s) affected	0.063 sec
6	03:07:50	CREATE TABLE customers (customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, la...	0 row(s) affected	0.031 sec
7	03:09:05	CREATE TABLE vehicles (vehicle_id INT PRIMARY KEY, customer_id INT NOT NULL, make VARCH...	0 row(s) affected	0.047 sec
8	03:09:52	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	0 row(s) affected. 2 warning(s): 1681 Integer display width is deprecated and will be removed in a future release...	0.032 sec
9	03:10:28	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	Error Code: 1050 Table 'products' already exists	0.032 sec
10	03:10:49	DROP TABLE products	0 row(s) affected	0.046 sec
11	03:10:55	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	0 row(s) affected. 1 warning(s): 1681 Integer display width is deprecated and will be removed in a future release...	0.031 sec
12	03:11:25	DROP TABLE products	0 row(s) affected	0.032 sec
13	03:11:29	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL...	0 row(s) affected	0.031 sec
14	03:13:15	CREATE TABLE premium_rates (rate_id INT PRIMARY KEY, product_id INT NOT NULL, vehicle_ca...	0 row(s) affected	0.047 sec

Quotes:

The screenshot shows a SQL IDE interface with a query editor, a schema navigator, and an output window. The query editor contains the following SQL code:

```
99 CREATE TABLE quotes (  
100     quote_id INT PRIMARY KEY,  
101     quote_no VARCHAR(25) NOT NULL UNIQUE,  
102     customer_id INT NOT NULL,  
103     vehicle_id INT NOT NULL,  
104     product_id INT NOT NULL,  
105     source_channel ENUM('B2B', 'B2C') NOT NULL,  
106     broker_id INT NOT NULL,  
107     sales_user_id INT NOT NULL,  
108     rate_id INT NOT NULL,  
109     add_on_selected INT NOT NULL DEFAULT 0,  
110     premium_before_tax DECIMAL(12,2) NOT NULL,  
111     tax_amount DECIMAL(12,2) NOT NULL,  
112     total_premium DECIMAL(12,2) NOT NULL,  
113     quote_status ENUM('DRAFT', 'REFERRED', 'APPROVED', 'PAID', 'EXPIRED') NOT NULL,  
114     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
115     CONSTRAINT fk_quotes_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id),  
116     CONSTRAINT fk_quotes_vehicle FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),  
117     CONSTRAINT fk_quotes_product FOREIGN KEY (product_id) REFERENCES products(product_id),  
118     CONSTRAINT fk_quotes_broker FOREIGN KEY (broker_id) REFERENCES brokers(broker_id),  
119     CONSTRAINT fk_quotes_sales_user FOREIGN KEY (sales_user_id) REFERENCES app_users(user_id),  
120     CONSTRAINT fk_quotes_rate FOREIGN KEY (rate_id) REFERENCES premium_rates(rate_id)  
121 );
```

The output window shows the execution results of the query. The first two rows show the creation of the database and the use of the database. The subsequent rows show the creation of various tables and the dropping of the 'products' table. The final row shows the creation of the 'quotes' table.

#	Time	Action	Message	Duration / Fetch
15	03:15:00	CREATE TABLE quotes (quote_id INT PRIMARY KEY, quote_no VARCHAR(25) NOT NULL UNIQUE...	Error Code: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL s...	0.000 sec
16	03:15:50	CREATE TABLE quotes (quote_id INT PRIMARY KEY, quote_no VARCHAR(25) NOT NULL UNIQUE...	0 row(s) affected	0.125 sec

Payments:

The screenshot shows a SQL IDE interface with the 'payments' table creation script in the main editor. The script defines columns: `payment_id` (INT PRIMARY KEY), `quote_id` (INT NOT NULL UNIQUE), `payment_method` (ENUM('CARD', 'BANK_TRANSFER', 'WALLET', 'BROKER_DEBIT') NOT NULL), `paid_currency` (CHAR(3) NOT NULL), `paid_amount` (DECIMAL(12,2) NOT NULL), `exchange_rate_to_inr` (DECIMAL(10,4) NOT NULL), `paid_amount_inr` (DECIMAL(12,2) NOT NULL), `payment_status` (ENUM('PENDING', 'SUCCESS', 'FAILED') NOT NULL), `paid_at` (DATETIME NULL), and a foreign key constraint `fk_payments_quote` referencing `quotes(quote_id)`.

The Output window shows the execution log for the 'payments' table creation. The log entry for the 'payments' table is as follows:

#	Time	Action	Message	Duration / Fetch
17	03:18:43	CREATE TABLE payments (payment_id INT PRIMARY KEY, quote_id INT NOT NULL UNIQUE, pay...	0 row(s) affected	0.047 sec

Policies:

The screenshot shows a SQL IDE interface with the 'policies' table creation script in the main editor. The script defines columns: `policy_id` (INT PRIMARY KEY), `policy_no` (VARCHAR(20) NOT NULL UNIQUE), `quote_id` (INT NOT NULL UNIQUE), `payment_id` (INT NOT NULL UNIQUE), `issue_date` (DATE NOT NULL), `start_date` (DATE NOT NULL), `end_date` (DATE NOT NULL), `approval_user_id` (INT NOT NULL), `policy_status` (ENUM('PENDING', 'ACTIVE', 'CANCELLED', 'LAPSED') NOT NULL), `debit_note_no` (VARCHAR(30) NULL), `credit_note_no` (VARCHAR(30) NULL), and three foreign key constraints: `fk_policies_quote` (referencing `quotes(quote_id)`), `fk_policies_payment` (referencing `payments(payment_id)`), and `fk_policies_approver` (referencing `app_users(user_id)`).

The Output window shows the execution log for the 'policies' table creation. The log entry for the 'policies' table is as follows:

#	Time	Action	Message	Duration / Fetch
18	03:21:03	CREATE TABLE policies (policy_id INT PRIMARY KEY, policy_no VARCHAR(20) NOT NULL UNIQUE, quote_id INT NOT NULL UNIQUE, payment_id INT NOT NULL UNIQUE, issue_date DATE NOT NULL, start_date DATE NOT NULL, end_date DATE NOT NULL, approval_user_id INT NOT NULL, policy_status ENUM('PENDING', 'ACTIVE', 'CANCELLED', 'LAPSED') NOT NULL, debit_note_no VARCHAR(30) NULL, credit_note_no VARCHAR(30) NULL, CONSTRAINT fk_policies_quote FOREIGN KEY (quote_id) REFERENCES quotes(quote_id), CONSTRAINT fk_policies_payment FOREIGN KEY (payment_id) REFERENCES payments(payment_id), CONSTRAINT fk_policies_approver FOREIGN KEY (approval_user_id) REFERENCES app_users(user_id))	0 row(s) affected	0.062 sec

Inserting Data to All the tables:

Regions:

Query 1

```
motor_insurance_assignment
(6, 'NorthEast', 'Assam', 'Gawahati', 'ACTIVE'),
(7, 'SouthWest', 'Karnataka', 'Bengaluru', 'ACTIVE'),
(8, 'NorthWest', 'Rajasthan', 'Jaipur', 'ACTIVE'),
(9, 'Coastal', 'Kerala', 'Kochi', 'ACTIVE'),
(10, 'Metro', 'Telangana', 'Hyderabad', 'ACTIVE');
SELECT * FROM regions;
```

Result Grid

region_id	region_name	state_name	city_name	status
1	North	Dehi	New Dehi	ACTIVE
2	South	Tamil Nadu	Chennai	ACTIVE
3	West	Maharashtra	Mumbai	ACTIVE
4	East	West Bengal	Kolkata	ACTIVE
5	Central	Madhya Pradesh	Bhopal	ACTIVE
6	NorthEast	Assam	Gawahati	ACTIVE
7	SouthWest	Karnataka	Bengaluru	ACTIVE
8	NorthWest	Rajasthan	Jaipur	ACTIVE
9	Coastal	Kerala	Kochi	ACTIVE
10	Metro	Telangana	Hyderabad	ACTIVE

regions 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
8	03:09:52	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT N...	0 row(s) affected. 2 warnings(s): 1681 Integer display width is deprecated and will be removed in a future rel...	0.032 sec
9	03:10:28	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT N...	Error Code 1050. Table 'products' already exists	0.032 sec
10	03:10:49	DROP TABLE products	0 row(s) affected	0.046 sec
11	03:10:55	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT N...	0 row(s) affected. 1 warning(s): 1681 Integer display width is deprecated and will be removed in a future rel...	0.031 sec
12	03:11:25	DROP TABLE products	0 row(s) affected	0.032 sec
13	03:11:29	CREATE TABLE products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT N...	0 row(s) affected	0.031 sec

App_users:

Query 1

```
SELECT * FROM app_users;
```

Result Grid

user_id	full_name	role	email	phone	region_id	manager_user_id	status	created_at
1	Asha Nair	ADMIN	asha.admin@motoramate.com	9000000001	1		ACTIVE	2026-01-01 09:00:00
2	Umar Khan	UNDERWRITER	umar.uw@motoramate.com	9000000002	2		ACTIVE	2026-01-01 09:05:00
3	Omkar Joshi	OPERATIONAL	omkar.ops@motoramate.com	9000000003	1	1	ACTIVE	2026-01-01 09:10:00
4	Priya Desai	OPERATIONAL	priya.ops@motoramate.com	9000000004	3	1	ACTIVE	2026-01-01 09:15:00
5	Rahul Verma	BROKER	rahul.broker@motoramate.com	9000000005	1	3	ACTIVE	2026-01-02 10:00:00
6	Neha Shah	BROKER	neha.broker@motoramate.com	9000000006	3	4	ACTIVE	2026-01-02 10:05:00
7	Kiran Gupta	SALES_AGENT	kiran.sales@motoramate.com	9000000007	1	5	ACTIVE	2026-01-02 10:10:00
8	Sana Iqbal	SALES_AGENT	sana.sales@motoramate.com	9000000008	1	5	ACTIVE	2026-01-02 10:15:00
9	Vikram Rao	SALES_AGENT	vikram.sales@motoramate.com	9000000009	3	6	ACTIVE	2026-01-02 10:20:00
10	Isha Thomas	SALES_AGENT	isha.sales@motoramate.com	9000000010	3	6	ACTIVE	2026-01-02 10:25:00

app_users 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
18	03:21:03	CREATE TABLE policies (policy_id INT PRIMARY KEY, policy_no VARCHAR(30) NOT NULL UNI...	0 row(s) affected	0.062 sec
19	09:06:01	INSERT INTO regions (region_id, region_name, state_name, city_name, status) VALUES (1, 'North', 'Dehi...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.031 sec
20	09:10:49	SELECT * FROM regions LIMIT 0. 1000	10 row(s) returned	0.000 sec / 0.000 sec
21	09:11:59	INSERT INTO app_users (user_id, full_name, role, email, phone, region_id, manager_user_id, status, crea...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
22	09:12:27	SELECT * FROM app_users LIMIT 0. 1000	10 row(s) returned	0.000 sec / 0.000 sec

Brokers:

Query 1

```
motor_insurance_assignment
(6, 'BRK-N001', 'NorthEast Secure Brokers', 'Tenzing Das', 6, NULL, 86000.00, 9.50, 'ACTIVE', '2026-01-03 09:25:00'),
(7, 'BRK-S001', 'SouthWest Motor Brokers', 'Naveen Rao', 7, NULL, 134000.00, 10.00, 'ACTIVE', '2026-01-03 09:30:00'),
(8, 'BRK-N001', 'NorthWest Policy Desk', 'Meera Solanki', 8, NULL, 91000.00, 9.25, 'ACTIVE', '2026-01-03 09:35:00'),
(9, 'BRK-C501', 'Coastal Motor Intermediaries', 'Aiden Mathew', 9, NULL, 150000.00, 10.75, 'ACTIVE', '2026-01-03 09:40:00'),
(10, 'BRK-M001', 'Metro Elite Brokers', 'Farah Ali', 10, NULL, 215000.00, 11.50, 'ACTIVE', '2026-01-03 09:45:00');
SELECT * FROM brokers;
```

Result Grid

broker_id	broker_code	broker_org_name	contact_name	region_id	linked_user_id	prepared_balance	commission_pct	status	created_at
1	BRK-N001	Monarch North Brokers	Raj Mehta	1	5	250000.00	12.50	ACTIVE	2026-01-03 09:00:00
2	BRK-W001	Apex West Insurance	Kavita Patel	3	6	175000.00	11.00	ACTIVE	2026-01-03 09:05:00
3	BRK-S001	South Shield Brokers	Arun Kumar	2		142500.00	10.50	ACTIVE	2026-01-03 09:10:00
4	BRK-E001	EastLine Risk Partners	Ritu Sen	4		98000.00	9.75	ACTIVE	2026-01-03 09:15:00
5	BRK-C001	Central Cover House	Sameer Jain	5		120000.00	10.25	ACTIVE	2026-01-03 09:20:00
6	BRK-N001	NorthEast Secure Brokers	Tenzing Das	6		86000.00	9.50	ACTIVE	2026-01-03 09:25:00
7	BRK-S001	SouthWest Motor Brokers	Naveen Rao	7		134000.00	10.00	ACTIVE	2026-01-03 09:30:00
8	BRK-M001	NorthWest Policy Desk	Meera Solanki	8		91000.00	9.25	ACTIVE	2026-01-03 09:35:00

brokers 3 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
17	03:18:43	CREATE TABLE payments (payment_id INT PRIMARY KEY, quote_id INT NOT NULL UNIQUE, ...	0 row(s) affected	0.047 sec
18	03:21:03	CREATE TABLE policies (policy_id INT PRIMARY KEY, policy_no VARCHAR(30) NOT NULL UNI...	0 row(s) affected	0.062 sec
19	09:06:01	INSERT INTO regions (region_id, region_name, state_name, city_name, status) VALUES (1, 'North', 'Dehi...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.031 sec
20	09:10:49	SELECT * FROM regions LIMIT 0. 1000	10 row(s) returned	0.000 sec / 0.000 sec
21	09:11:59	INSERT INTO app_users (user_id, full_name, role, email, phone, region_id, manager_user_id, status, crea...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
22	09:12:27	SELECT * FROM app_users LIMIT 0. 1000	10 row(s) returned	0.000 sec / 0.000 sec
23	09:13:18	INSERT INTO brokers (broker_id, broker_code, broker_org_name, contact_name, region_id, linked_user...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
24	09:13:39	SELECT * FROM brokers LIMIT 0. 1000	10 row(s) returned	0.000 sec / 0.000 sec

Customers:

The screenshot shows a database management tool interface. The top pane displays a query for 'motor_insurance_assignment' with a limit of 1000 rows. The query is: `select * FROM customers;`. The bottom pane shows the results of the query, which is a table with columns: customer_id, first_name, last_name, dob, gender, email, phone, national_id, city_name, state_name, and created_at. The results are displayed in a grid format. On the right side, there is a sidebar with a 'SQL Additions' section and a 'Context Help' section. The 'Context Help' section contains a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

customer_id	first_name	last_name	dob	gender	email	phone	national_id	city_name	state_name	created_at
1	Arjun	Mahotra	1990-06-10	M	arjun.mahotra@gmail.com	9810000001	IND1000001	New Delhi	Delhi	2026-01-03 10:00:00
2	Nisha	Reddy	1988-09-15	F	nisha.reddy@gmail.com	9810000002	IND1000002	Hyderabad	Telangana	2026-01-03 10:00:00
3	Daniel	Fernandes	1985-03-21	M	daniel.fernandes@gmail.com	9810000003	IND1000003	Bengaluru	Karnataka	2026-01-03 10:00:00
4	Mital	Sen	1995-12-02	F	mital.sen@gmail.com	9810000004	IND1000004	Kolkata	West Bengal	2026-01-03 10:00:00
5	Rohit	Kapoor	1992-07-19	M	rohit.kapoor@gmail.com	9810000005	IND1000005	Mumbai	Maharashtra	2026-01-03 10:00:00
6	Ananya	Iyer	1991-11-08	F	ananya.iyer@gmail.com	9810000006	IND1000006	Chennai	Tamil Nadu	2026-01-03 10:00:00
7	Suresh	Patel	1987-04-12	M	suresh.patel@gmail.com	9810000007	IND1000007	Indore	Madhya Pradesh	2026-01-03 10:00:00
8	Rishika	Gosh	1993-01-18	F	rishika.gosh@gmail.com	9810000008	IND1000008	Bangalore	Karnataka	2026-01-03 10:00:00

Vehicles:

The screenshot shows a database management tool interface. The top pane displays a query for 'motor_insurance_assignment' with a limit of 1000 rows. The query is: `SELECT * FROM vehicles;`. The bottom pane shows the results of the query, which is a table with columns: vehicle_id, customer_id, make, model, model_year, body_type, vehicle_category, registration_no, sum_insured, status, and created_at. The results are displayed in a grid format. On the right side, there is a sidebar with a 'SQL Additions' section and a 'Context Help' section. The 'Context Help' section contains a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

vehicle_id	customer_id	make	model	model_year	body_type	vehicle_category	registration_no	sum_insured	status	created_at
1	1	Honda	City	2022	Sedan	PRIVATE	DL01AB1234	850000.00	ACTIVE	2026-01-03 11:00:00
2	2	Maruti	Swift Dzire	2021	Sedan	TAXI	TS09CD2345	720000.00	ACTIVE	2026-01-03 11:05:00
3	3	Tata	Ace	2020	Mini Truck	COMMERCIAL	KA09EP3456	540000.00	ACTIVE	2026-01-03 11:10:00
4	4	Royal Enfield	Classic 350	2023	Motorbike	MOTORCYCLE	WB12QH4567	250000.00	ACTIVE	2026-01-03 11:15:00
5	5	BMW	M340i	2024	Sedan	SPORTS	MH14J5678	4200000.00	ACTIVE	2026-01-03 11:20:00
6	6	Hyundai	Creta	2022	SUV	PRIVATE	TN10KL6789	1450000.00	ACTIVE	2026-01-03 11:25:00
7	7	Mahindra	Bolevo	2019	SUV	COMMERCIAL	HP04BH7890	880000.00	ACTIVE	2026-01-03 11:30:00
8	8	Toyota	Innova	2021	MUV	TAXI	RJ19OP8901	1600000.00	ACTIVE	2026-01-03 11:35:00
9	9	Kia	Seltos	2023	SUV	PRIVATE	KR17NR9012	1750000.00	ACTIVE	2026-01-03 11:40:00

Products:

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

product_id	product_name	cover_type	has_roadside_assist	has_zero_dep	base_tax_pct	currency_code	status
1	Comprehensive Basic	COMPREHENSIVE	1	0	18.00	INR	ACTIVE
2	TPL Standard	TPL	0	0	18.00	INR	ACTIVE
3	Comprehensive Plus	COMPREHENSIVE	1	1	18.00	INR	ACTIVE
4	Taxi Shield	TPL	1	0	18.00	INR	ACTIVE
5	Sports Max Comprehensive	COMPREHENSIVE	1	1	18.00	INR	ACTIVE
6	Private Value Comprehensive	COMPREHENSIVE	0	0	18.00	INR	ACTIVE
7	Commercial Guard	TPL	0	0	18.00	INR	ACTIVE
8	Taxi Premier	COMPREHENSIVE	1	0	18.00	INR	ACTIVE

products 6 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
23	09:13:18	INSERT INTO brokers (broker_id, broker_code, broker_name, contact_name, region_id, linked_user...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
24	09:13:39	SELECT * FROM brokers LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
25	09:14:47	INSERT INTO customers (customer_id, first_name, last_name, dob, gender, email, phone, national_id, city...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
26	09:15:07	select * FROM customers LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
27	09:15:40	INSERT INTO vehicles (vehicle_id, customer_id, make, model, model_year, body_type, vehicle_category...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
28	09:15:55	SELECT * FROM vehicles LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
29	09:16:43	INSERT INTO products (product_id, product_name, cover_type, has_roadside_assist, has_zero_dep, bas...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
30	09:17:09	SELECT * FROM products LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Premium_Rates:

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

rate_id	product_id	vehicle_category	risk_band	base_rate_pct	add_on_rate_pct	min_premium	valid_from	valid_to	status
1	1	PRIVATE	LOW	1.45	0.35	7000.00	2026-01-01	2026-12-31	ACTIVE
2	2	TAXI	MEDIUM	1.90	0.30	6500.00	2026-01-01	2026-12-31	ACTIVE
3	3	COMMERCIAL	HIGH	2.40	0.45	9000.00	2026-01-01	2026-12-31	ACTIVE
4	4	MOTORCYCLE	LOW	1.10	0.30	2500.00	2026-01-01	2026-12-31	ACTIVE
5	5	SPORTS	HIGH	2.90	0.60	12000.00	2026-01-01	2026-12-31	ACTIVE
6	6	PRIVATE	MEDIUM	1.70	0.15	7800.00	2026-01-01	2026-12-31	ACTIVE
7	7	COMMERCIAL	MEDIUM	2.10	0.20	8200.00	2026-01-01	2026-12-31	ACTIVE
8	8	TAXI	HIGH	2.35	0.30	8700.00	2026-01-01	2026-12-31	ACTIVE

premium_rates 7 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
25	09:14:47	INSERT INTO customers (customer_id, first_name, last_name, dob, gender, email, phone, national_id, city...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
26	09:15:07	select * FROM customers LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
27	09:15:40	INSERT INTO vehicles (vehicle_id, customer_id, make, model, model_year, body_type, vehicle_category...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
28	09:15:55	SELECT * FROM vehicles LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
29	09:16:43	INSERT INTO products (product_id, product_name, cover_type, has_roadside_assist, has_zero_dep, bas...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
30	09:17:09	SELECT * FROM products LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
31	09:18:05	INSERT INTO premium_rates (rate_id, product_id, vehicle_category, risk_band, base_rate_pct, add_on_ra...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
32	09:18:29	SELECT * FROM premium_rates LIMIT 0, 1000	10 row(s) returned	0.016 sec / 0.000 sec

Quotes:

The screenshot displays a database management interface with a SQL query window and a results pane. The query window shows a SQL statement for inserting data into the 'quotes' table. The results pane shows the execution output, including a table of inserted records and a log of actions.

SQL Query:

```
INSERT INTO quotes (quote_id, quote_no, customer_id, vehicle_id, product_id, source_channel, broker_id, sales_user_id, rate, addon_selected, premium_before_tax, tax_an) VALUES (8, 'Q-2026-0008', 8, 8, 8, 'B2C', NULL, 8, 1, 19500.00, 3510.00, 23010.00, 'DRAFT', '2026-01-06 10:00:00'), (9, 'Q-2026-0009', 9, 9, 9, 'B2B', 6, 10, 9, 1, 17400.00, 3132.00, 20532.00, 'PAID', '2026-01-07 12:00:00'), (10, 'Q-2026-0010', 10, 10, 10, 'B2B', 2, 9, 10, 0, 3900.00, 702.00, 4602.00, 'PAID', '2026-01-07 13:00:00');
```

Result Grid:

quote_id	quote_no	customer_id	vehicle_id	product_id	source_channel	broker_id	sales_user_id	rate	addon_selected	premium_before_tax	tax_an
1	Q-2026-0001	1	1	1	B2B	1	7	1	1	12000.00	2160.00
2	Q-2026-0002	2	2	2	B2C	8	8	2	0	8500.00	1530.00
3	Q-2026-0003	3	3	3	B2B	2	9	3	1	14500.00	2610.00
4	Q-2026-0004	4	4	4	B2B	3	8	4	0	3200.00	576.00
5	Q-2026-0005	5	5	5	B2C	10	5	1	1	28000.00	5040.00
6	Q-2026-0006	6	6	6	B2B	4	9	6	1	15800.00	2944.00
7	Q-2026-0007	7	7	7	B2B	5	7	7	0	11200.00	2016.00
8	Q-2026-0008	8	8	8	B2C	8	8	8	1	19500.00	3510.00

Action Output:

#	Time	Action	Message	Duration / Fetch
27	09:15:40	INSERT INTO vehicles (vehicle_id, customer_id, make, model, model_year, body_type, vehicle_category, ...)	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
28	09:15:55	SELECT * FROM vehicles LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
29	09:16:43	INSERT INTO products (product_id, product_name, cover_type, has_roadside_assist, has_zero_dep, bas...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
30	09:17:09	SELECT * FROM products LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
31	09:18:05	INSERT INTO premium_rates (rate_id, product_id, vehicle_category, risk_band, base_rate_pct, addon_ra...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
32	09:18:29	SELECT * FROM premium_rates LIMIT 0, 1000	10 row(s) returned	0.016 sec / 0.000 sec
33	09:19:28	INSERT INTO quotes (quote_id, quote_no, customer_id, vehicle_id, product_id, source_channel, broker_...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
34	09:19:53	SELECT * FROM quotes LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Payments:

The screenshot displays a database management interface with a SQL query window and a results pane. The query window shows a SQL statement for inserting data into the 'payments' table. The results pane shows the execution output, including a table of inserted records and a log of actions.

SQL Query:

```
INSERT INTO payments (payment_id, quote_id, payment_method, paid_currency, paid_amount, exchange_rate_to_inr, paid_amount_inr, payment_status, paid_at) VALUES (8, 'CARD', 'INR', 0.00, 1.0000, 0.00, 'PENDING', NULL), (9, 'WALLET', 'AED', 900.00, 22.8133, 20532.00, 'SUCCESS', '2026-01-07 14:30:00'), (10, 'CARD', 'INR', 4602.00, 1.0000, 4602.00, 'SUCCESS', '2026-01-07 15:00:00');
```

Result Grid:

payment_id	quote_id	payment_method	paid_currency	paid_amount	exchange_rate_to_inr	paid_amount_inr	payment_status	paid_at
1	1	BROKER_DEBIT	INR	14160.00	1.0000	14160.00	SUCCESS	2026-01-03 14:00:00
2	2	CARD	USD	120.00	83.5833	10030.00	SUCCESS	2026-01-04 14:30:00
3	3	BANK_TRANSFER	INR	17110.00	1.0000	17110.00	SUCCESS	2026-01-04 15:00:00
4	4	CARD	INR	0.00	1.0000	0.00	PENDING	2026-01-06 10:00:00
5	5	WALLET	INR	0.00	1.0000	0.00	PENDING	2026-01-06 10:00:00
6	6	BROKER_DEBIT	INR	18644.00	1.0000	18644.00	SUCCESS	2026-01-06 13:00:00
7	7	BANK_TRANSFER	INR	13216.00	1.0000	13216.00	SUCCESS	2026-01-06 13:30:00
8	8	CARD	INR	0.00	1.0000	0.00	PENDING	2026-01-07 14:30:00
9	9	WALLET	AED	900.00	22.8133	20532.00	SUCCESS	2026-01-07 14:30:00
10	10	CARD	INR	4602.00	1.0000	4602.00	SUCCESS	2026-01-07 15:00:00

Action Output:

#	Time	Action	Message	Duration / Fetch
29	09:16:43	INSERT INTO products (product_id, product_name, cover_type, has_roadside_assist, has_zero_dep, bas...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.000 sec
30	09:17:09	SELECT * FROM products LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
31	09:18:05	INSERT INTO premium_rates (rate_id, product_id, vehicle_category, risk_band, base_rate_pct, addon_ra...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
32	09:18:29	SELECT * FROM premium_rates LIMIT 0, 1000	10 row(s) returned	0.016 sec / 0.000 sec
33	09:19:28	INSERT INTO quotes (quote_id, quote_no, customer_id, vehicle_id, product_id, source_channel, broker_...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
34	09:19:53	SELECT * FROM quotes LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec
35	09:20:55	INSERT INTO payments (payment_id, quote_id, payment_method, paid_currency, paid_amount, exchange...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
36	09:21:22	SELECT * FROM payments LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 sec

Policies:

Query 1: motor_insurance_assignment

```
315 (8, 'POL-2026-0008', 8, NULL, '2026-01-06', '2026-01-07', '2027-01-06', 2, 'PENDING', NULL, NULL),
316 (9, 'POL-2026-0009', 9, 9, '2026-01-07', '2026-01-08', '2027-01-07', 3, 'ACTIVE', 'DN-1009', 'CN-1009'),
317 (10, 'POL-2026-0010', 10, 10, '2026-01-07', '2026-01-08', '2026-08-31', 3, 'LAPSED', 'DN-1010', 'CN-1010');
318
319
320
321 • SELECT * FROM policies;
```

Result Grid:

policy_id	policy_no	quote_id	payment_id	issue_date	start_date	end_date	approval_user_id	policy_status	debit_note_no	credit_note_no
1	POL-2026-0001	1	1	2026-01-03	2026-01-04	2027-01-03	3	ACTIVE	DN-1001	CN-1001
2	POL-2026-0002	2	2	2026-01-04	2026-01-05	2027-01-04	3	ACTIVE	DN-1002	CN-1002
3	POL-2026-0003	3	3	2026-01-04	2026-01-05	2027-01-04	4	ACTIVE	DN-1003	CN-1003
4	POL-2026-0004	4	4	2026-01-05	2026-01-06	2027-01-05	2	PENDING	DN-1004	CN-1004
5	POL-2026-0005	5	5	2026-01-05	2026-01-06	2027-01-05	2	PENDING	DN-1005	CN-1005
6	POL-2026-0006	6	6	2026-01-06	2026-01-07	2027-01-06	4	ACTIVE	DN-1006	CN-1006
7	POL-2026-0007	7	7	2026-01-06	2026-01-07	2026-10-31	4	CANCELLED	DN-1007	CN-1007
8	POL-2026-0008	8	8	2026-01-06	2026-01-07	2027-01-06	2	PENDING	DN-1008	CN-1008
9	POL-2026-0009	9	9	2026-01-07	2026-01-08	2027-01-07	3	ACTIVE	DN-1009	CN-1009

Output:

#	Time	Action	Message	Duration
31	09:18:05	INSERT INTO premium_rates (rate_id, product_id, vehicle_category, risk_band, base_rate_pct, addon_ra...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.016 sec
32	09:18:29	SELECT * FROM premium_rates LIMIT 0, 1000	10 row(s) returned	0.016 sec
33	09:19:28	INSERT INTO quotes (quote_id, quote_no, customer_id, vehicle_id, product_id, source_channel, broker_...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
34	09:19:53	SELECT * FROM quotes LIMIT 0, 1000	10 row(s) returned	0.000 sec
35	09:20:55	INSERT INTO payments (payment_id, quote_id, payment_method, paid_currency, paid_amount, exchang...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
36	09:21:22	SELECT * FROM payments LIMIT 0, 1000	10 row(s) returned	0.000 sec
37	09:22:03	INSERT INTO policies (policy_id, policy_no, quote_id, payment_id, issue_date, start_date, end_date, app...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
38	09:22:37	SELECT * FROM policies LIMIT 0, 1000	10 row(s) returned	0.000 sec

QUESTIONS:

Q1) List all customers

SELECT * FROM customers;

Query 1: motor_insurance_assignment

```
340 -- *****
341
342
343 -- Q1) List all customers.
344
345 • SELECT *
346 FROM customers;
```

Result Grid:

customer_id	first_name	last_name	dob	gender	email	phone	national_id	city_name	state_name	created_at
1	Arjun	Malhotra	1990-06-10	M	arjun.malhotra@email.com	9810000001	NID100001	New Delhi	Delhi	2026-01-03 10:10:10
2	Nisha	Reddy	1988-09-15	F	nisha.reddy@email.com	9810000002	NID100002	Hyderabad	Telangana	2026-01-03 10:10:10
3	Daniel	Fernandes	1985-03-21	M	daniel.fernandes@email.com	9810000003	NID100003	Bengaluru	Karnataka	2026-01-03 10:10:10
4	Mitali	Sen	1995-12-02	F	mitali.sen@email.com	9810000004	NID100004	Kolkata	West Bengal	2026-01-03 10:10:10
5	Rohit	Kapoor	1992-07-19	M	rohit.kapoor@email.com	9810000005	NID100005	Mumbai	Maharashtra	2026-01-03 10:10:10
6	Ananya	Iyer	1991-11-08	F	ananya.iyer@email.com	9810000006	NID100006	Chennai	Tamil Nadu	2026-01-03 10:10:10
7	Suresh	Patel	1987-04-12	M	suresh.patel@email.com	9810000007	NID100007	Indore	Madhya Pradesh	2026-01-03 10:10:10
8	Ravi	Gupta	1993-01-28	F	ravi.gupta@email.com	9810000008	NID100008	Jaipur	Rajasthan	2026-01-03 10:10:10

Output:

#	Time	Action	Message	Duration / Fetch
32	09:18:29	SELECT * FROM premium_rates LIMIT 0, 1000	10 row(s) returned	0.016 sec / 0.000 s
33	09:19:28	INSERT INTO quotes (quote_id, quote_no, customer_id, vehicle_id, product_id, source_channel, broker_...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
34	09:19:53	SELECT * FROM quotes LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 s
35	09:20:55	INSERT INTO payments (payment_id, quote_id, payment_method, paid_currency, paid_amount, exchang...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
36	09:21:22	SELECT * FROM payments LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 s
37	09:22:03	INSERT INTO policies (policy_id, policy_no, quote_id, payment_id, issue_date, start_date, end_date, app...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
38	09:22:37	SELECT * FROM policies LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 s
39	09:27:30	SELECT * FROM customers LIMIT 0, 1000	10 row(s) returned	0.000 sec / 0.000 s

Q2) Show active brokers with prepaid balance above 100000.

```
SELECT broker_code, broker_org_name, prepaid_balance
FROM brokers
WHERE status = 'ACTIVE' AND prepaid_balance > 100000
ORDER BY prepaid_balance DESC;
```

The screenshot shows a database management tool interface. The top menu bar includes File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows a Schemas tree with folders for college, company, Tables, Views, Stored Procedures, Functions, and dmart. The main window displays a SQL query in a text editor, with line numbers 347 to 353. The query is:
347
348 -- Q2) Show active brokers with prepaid balance above 100000.
349
350 SELECT broker_code, broker_org_name, prepaid_balance
351 FROM brokers
352 WHERE status = 'ACTIVE' AND prepaid_balance > 100000
353 ORDER BY prepaid_balance DESC;
Below the query editor, the 'Result Grid' shows a table with 3 columns: broker_code, broker_org_name, and prepaid_balance. The table contains 8 rows of data. The bottom section shows the 'Output' pane with a list of actions and their messages, including SELECT and INSERT statements and their execution status.

broker_code	broker_org_name	prepaid_balance
BRK-H001	Monarch North Brokers	250000.00
BRK-M001	Metro Elite Brokers	215000.00
BRK-W001	Apex West Insurance	175000.00
BRK-CS01	Coastal Motor Intermediaries	150000.00
BRK-S001	South Shield Brokers	142500.00
BRK-SW01	SouthWest Motor Brokers	134000.00
BRK-C001	Central Cover House	120000.00

#	Time	Action	Message
34	09:19:53	SELECT * FROM quotes LIMIT 0, 1000	10 row(s) returned
35	09:20:55	INSERT INTO payments (payment_id, quote_id, payment_method, paid_currency, paid_amount, exchange...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
36	09:21:22	SELECT * FROM payments LIMIT 0, 1000	10 row(s) returned
37	09:22:03	INSERT INTO policies (policy_id, policy_no, quote_id, payment_id, issue_date, start_date, end_date, app...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
38	09:22:37	SELECT * FROM policies LIMIT 0, 1000	10 row(s) returned
39	09:27:30	SELECT * FROM customers LIMIT 0, 1000	10 row(s) returned
40	09:30:15	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned
41	11:46:24	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned

Q3) Show top 5 quotes by total premium.

```
SELECT quote_no, total_premium, quote_status
FROM quotes
ORDER BY total_premium DESC
LIMIT 5;
```

Query 1: motor_insurance_assignment

```

353 ORDER BY prepaid_balance DESC;
354
355 -- Q3) Show top 5 quotes by total premium.
356 SELECT quote_no, total_premium, quote_status
357 FROM quotes
358 ORDER BY total_premium DESC
359 LIMIT 5;

```

quote_no	total_premium	quote_status
Q-2026-0005	33040.00	APPROVED
Q-2026-0008	23010.00	DRAFT
Q-2026-0009	20532.00	PAID
Q-2026-0006	18644.00	PAID
Q-2026-0003	17110.00	PAID

quotes 14 x

Output

#	Time	Action	Message
38	09:22:37	SELECT * FROM policies LIMIT 0, 1000	10 row(s) returned
39	09:27:30	SELECT * FROM customers LIMIT 0, 1000	10 row(s) returned
40	09:30:15	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned
41	11:46:24	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned
42	11:47:47	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to yo
43	11:47:49	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to yo
44	11:48:16	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to yo
45	11:48:25	SELECT quote_no, total_premium, quote_status FROM quotes ORDER BY total_premium DESC LIMIT 5	5 row(s) returned

Q4) Show distinct vehicle categories in the system.

SELECT DISTINCT vehicle_category

FROM vehicles

ORDER BY vehicle_category;

Query 1: motor_insurance_assignment

```

359 LIMIT 5;
360
361 -- Q4) Show distinct vehicle categories in the system.
362 SELECT DISTINCT vehicle_category
363 FROM vehicles
364 ORDER BY vehicle_category;
365

```

vehicle_category
PRIVATE
COMMERCIAL
TAXI
MOTORCYCLE
SPORTS

vehicles 15 x

Output

#	Time	Action	Message
39	09:27:30	SELECT * FROM customers LIMIT 0, 1000	10 row(s) returned
40	09:30:15	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned
41	11:46:24	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned
42	11:47:47	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that correspo
43	11:47:49	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that correspo
44	11:48:16	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that correspo
45	11:48:25	SELECT quote_no, total_premium, quote_status FROM quotes ORDER BY total_premium DESC LIMIT 5	5 row(s) returned
46	11:48:53	SELECT DISTINCT vehicle_category FROM vehicles ORDER BY vehicle_category LIMIT 0, 1000	5 row(s) returned

Q5) Count total quotes and paid quotes.

SELECT

COUNT(*) AS total_quotes,

SUM(CASE WHEN quote_status = 'PAID' THEN 1 ELSE 0 END) AS paid_quotes

FROM quotes;

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
-- Q5) Count total quotes and paid quotes.  
SELECT  
    COUNT(*) AS total_quotes,  
    SUM(CASE WHEN quote_status = 'PAID' THEN 1 ELSE 0 END) AS paid_quotes  
FROM quotes;
```

The results pane displays a table with two columns: **total_quotes** and **paid_quotes**. The first row shows the values 10 and 7 respectively.

total_quotes	paid_quotes
10	7

The bottom pane shows the execution log with the following entries:

#	Time	Action	Message
41	11:46:24	SELECT broker_code, broker_org_name, prepaid_balance FROM brokers WHERE status = 'ACTIVE' AN...	7 row(s) returned
42	11:47:47	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that correspor
43	11:47:49	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that correspor
44	11:48:16	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that correspor
45	11:48:25	SELECT quote_no, total_premium, quote_status FROM quotes ORDER BY total_premium DESC LIMIT 5	5 row(s) returned
46	11:48:53	SELECT DISTINCT vehicle_category FROM vehicles ORDER BY vehicle_category LIMIT 0, 1000	5 row(s) returned
47	11:50:16	SELECT COUNT(*) AS total_quotes, SUM(CASE WHEN quote_status = 'PAID' THEN 1 ELSE 0 EN...	1 row(s) returned
48	11:54:52	SELECT COUNT(*) AS total_quotes, SUM(CASE WHEN quote_status = 'PAID' THEN 1 ELSE 0 EN...	1 row(s) returned

Q6) Count users by role.

SELECT role, COUNT(*) AS user_count

FROM app_users

GROUP BY role

ORDER BY user_count DESC;

Query 1: motor_insurance_assignment

```

371
372 -- Q6) Count users by role.
373 • SELECT role, COUNT(*) AS user_count
374 FROM app_users
375 GROUP BY role
376 ORDER BY user_count DESC;
377

```

Result Grid

role	user_count
SALES_AGENT	4
OPERATIONAL	2
BROKER	2
ADMIN	1
UNDERWRITER	1

Result 18

Output

#	Time	Action	Message
42	11:47:47	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'SELECT quote_no, total_premium, quote_status FROM quotes ...'
43	11:47:49	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'SELECT quote_no, total_premium, quote_status FROM quotes ...'
44	11:48:16	Q3) Show top 5 quotes by total premium. SELECT quote_no, total_premium, quote_status FROM quotes ...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'SELECT quote_no, total_premium, quote_status FROM quotes ...'
45	11:48:25	SELECT quote_no, total_premium, quote_status FROM quotes ORDER BY total_premium DESC LIMIT 5	5 row(s) returned
46	11:48:53	SELECT DISTINCT vehicle_category FROM vehicles ORDER BY vehicle_category LIMIT 0, 1000	5 row(s) returned
47	11:50:16	SELECT COUNT(*) AS total_quotes, SUM(CASE WHEN quote_status = 'PAID' THEN 1 ELSE 0 END) AS paid_quotes FROM quotes	1 row(s) returned
48	11:54:52	SELECT COUNT(*) AS total_quotes, SUM(CASE WHEN quote_status = 'PAID' THEN 1 ELSE 0 END) AS paid_quotes FROM quotes	1 row(s) returned
49	11:57:33	SELECT role, COUNT(*) AS user_count FROM app_users GROUP BY role ORDER BY user_count DESC	5 row(s) returned

Q7) Average and max premium by product cover type.

SELECT

p.cover_type,

ROUND(AVG(q.total_premium), 2) AS avg_premium,

MAX(q.total_premium) AS max_premium

FROM quotes q

JOIN products p ON p.product_id = q.product_id

GROUP BY p.cover_type;

377
378 -- Q7) Average and max premium by product cover type.
379 • SELECT
380 p.cover_type,
381 ROUND(AVG(q.total_premium), 2) AS avg_premium,
382 MAX(q.total_premium) AS max_premium
383 FROM quotes q
384 JOIN products p ON p.product_id = q.product_id
385 GROUP BY p.cover_type;

cover_type	avg_premium	max_premium
COMPREHENSIVE	21082.67	33040.00
TPL	7906.00	13216.00

Result 21 x

Output

#	Time	Action	Message
50	11:59:30	SELECT p.cover_type, ROUND(AVG(q.total_premium), 2) AS avg_premium, MAX(q.total_premium) AS max_premium FROM quotes q JOIN...	2 row(s) returned
51	13:20:35	Q8) Join customer + vehicle + quote details. SELECT q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premium, q.quote_status	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server vers...
52	13:20:50	SELECT q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premium, q.quote_status	10 row(s) returned
53	13:21:38	SELECT p.cover_type, ROUND(AVG(q.total_premium), 2) AS avg_premium, MAX(q.total_premium) AS max_premium FROM quotes q JOIN...	2 row(s) returned

Q8) Join customer + vehicle + quote details.

SELECT

q.quote_no,

CONCAT(c.first_name, ' ', c.last_name) AS customer_name,

v.registration_no,

v.make,

v.model,

q.total_premium,

q.quote_status

FROM quotes q

JOIN customers c ON c.customer_id = q.customer_id

JOIN vehicles v ON v.vehicle_id = q.vehicle_id

ORDER BY q.quote_id;

cedures

```

386
387 -- Q8) Join customer + vehicle + quote details.
388 • SELECT
389     q.quote_no,
390     CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
391     v.registration_no,
392     v.make,
393     v.model,
394     q.total_premium,
395     q.quote_status
396 FROM quotes q
397 JOIN customers c ON c.customer_id = q.customer_id
398 JOIN vehicles v ON v.vehicle_id = q.vehicle_id
399 ORDER BY q.quote_id;
400

```

Result Grid

quote_no	customer_name	registration_no	make	model	total_premium	quote_status
Q-2026-0001	Arjun Malhotra	DL01AB1234	Honda	City	14160.00	PAID
Q-2026-0002	Neha Reddy	TS09CD2345	Maruti	Swift Dzire	10030.00	PAID
Q-2026-0003	Daniel Fernandes	KA05EF3456	Tata	Ace	17110.00	PAID
Q-2026-0004	Mital Sen	WB12GH4567	Royal Enfield	Classic 350	3776.00	REFERRED
Q-2026-0005	Rohit Kapoor	MH14IJ5678	BMW	M340i	33040.00	APPROVED
Q-2026-0006	Ananya Iyer	TN19KL6789	Hyundai	Creta	18644.00	PAID
Q-2026-0007	Suresh Patel	NP04MN7890	Marhndra	Bolero	13216.00	PAID
Q-2026-0008	Pooja Singh	RJ19OP8901	Toyota	Innova	23010.00	DRAFT
Q-2026-0009	Joseph Mathew	KL07QR9012	Kia	Seltos	20532.00	PAID
Q-2026-0010	Fatma Noor	TS08ST0123	Yamaha	R15	4602.00	PAID

Result 22 x

Output

#	Time	Action	Message
51	13:20:35	Q8) Join customer + vehicle + quote details. SELECT q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premium, q.quote_status FROM quotes q JOIN customers c ON c.customer_id = q.customer_id JOIN vehicles v ON v.vehicle_id = q.vehicle_id ORDER BY q.quote_id;	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version
52	13:20:50	SELECT q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premium, q.quote_status FROM quotes q JOIN customers c ON c.customer_id = q.customer_id JOIN vehicles v ON v.vehicle_id = q.vehicle_id ORDER BY q.quote_id;	10 row(s) returned
53	13:21:38	SELECT p.cover_type, ROUND(AVG(q.total_premium), 2) AS avg_premium, MAX(q.total_premium) AS max_premium FROM quotes q JOIN customers c ON c.customer_id = q.customer_id JOIN vehicles v ON v.vehicle_id = q.vehicle_id GROUP BY p.cover_type;	2 row(s) returned
54	13:22:34	SELECT q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premium, q.quote_status FROM quotes q JOIN customers c ON c.customer_id = q.customer_id JOIN vehicles v ON v.vehicle_id = q.vehicle_id ORDER BY q.quote_id;	10 row(s) returned

in

Q9) Left join brokers and quotes to show quote counts (including brokers with 0 quotes).

SELECT

b.broker_code,

b.broker_org_name,

COUNT(q.quote_id) AS quote_count

FROM brokers b

LEFT JOIN quotes q ON q.broker_id = b.broker_id

GROUP BY b.broker_id, b.broker_code, b.broker_org_name

ORDER BY quote_count DESC, b.broker_code;

disabled. I manually current ca toggle z

```

490
491 -- (9) Left join brokers and quotes to show quote counts (including brokers with 0 quotes).
492 * SELECT
493     b.broker_code,
494     b.broker_org_name,
495     COUNT(q.quote_id) AS quote_count
496 FROM brokers b
497 LEFT JOIN quotes q ON q.broker_id = b.broker_id
498 GROUP BY b.broker_id, b.broker_code, b.broker_org_name
499 ORDER BY quote_count DESC, b.broker_code

```

broker_code	broker_org_name	quote_count
BRK-W001	Apex West Insurance	2
BRK-C001	Central Cover House	1
BRK-E001	Eastline Risk Partners	1
BRK-N001	Monarch North Brokers	1
BRK-NE01	NorthEast Secure Brokers	1
BRK-S001	South Shield Brokers	1
BRK-C301	Coastal Motor Intermediaries	0
BRK-M001	Metro Elite Brokers	0
BRK-NW01	NorthWest Policy Desk	0
BRK-SW01	SouthWest Motor Brokers	0

Result 23 x

Output

#	Time	Action	Message
52	13:20:50	SELECT	q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premi... 10 row(s) returned
53	13:21:38	SELECT	p.cover_type, ROUND(AVG(q.total_premium), 2) AS avg_premium, MAX(q.total_premium) AS max_premium FROM quotes q JOIN... 2 row(s) returned
54	13:22:34	SELECT	q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premi... 10 row(s) returned
55	13:24:10	SELECT	b.broker_code, b.broker_org_name, COUNT(q.quote_id) AS quote_count FROM brokers b LEFT JOIN quotes q ON q.broker_id ... 10 row(s) returned

Q10) Find pending business actions (quote or payment or policy pending).

SELECT

q.quote_no,

q.quote_status,

pay.payment_status,

pol.policy_status

FROM quotes q

JOIN payments pay ON pay.quote_id = q.quote_id

JOIN policies pol ON pol.quote_id = q.quote_id

WHERE q.quote_status IN ('DRAFT', 'REFERRED', 'APPROVED')

OR pay.payment_status = 'PENDING'

OR pol.policy_status = 'PENDING'

ORDER BY q.quote_id;

411 -- Q10) Find pending business actions (quote or payment or policy pending).
 412 SELECT
 413 q.quote_no,
 414 q.quote_status,
 415 pay.payment_status,
 416 pol.policy_status
 417 FROM quotes q
 418 JOIN payments pay ON pay.quote_id = q.quote_id
 419 JOIN policies pol ON pol.quote_id = q.quote_id
 420 WHERE q.quote_status IN ('DRAFT', 'REFERRED', 'APPROVED')
 421 OR pay.payment_status = 'PENDING'
 422 OR pol.policy_status = 'PENDING'
 423 ORDER BY q.quote_id;
 424

quote_no	quote_status	payment_status	policy_status
Q-2026-0004	REFERRED	PENDING	PENDING
Q-2026-0005	APPROVED	PENDING	PENDING
Q-2026-0008	DRAFT	PENDING	PENDING

Result 24 x

Output

#	Time	Action	Message
53	13:21:38	SELECT	p.cover_type, ROUND(AVG(q.total_premium), 2) AS avg_premium, MAX(q.total_premium) AS max_premium FROM quotes q JOIN ... 2 row(s) returned
54	13:22:34	SELECT	q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premi... 10 row(s) returned
55	13:24:10	SELECT	b.broker_code, b.broker_org_name, COUNT(q.quote_id) AS quote_count FROM brokers b LEFT JOIN quotes q ON q.broker_id ... 10 row(s) returned
56	13:25:09	SELECT	q.quote_no, q.quote_status, pay.payment_status, pol.policy_status FROM quotes q JOIN payments pay ON pay.quote_id = q ... 3 row(s) returned

Q11) Region-wise policy count and premium collection using sales user region.

SELECT

r.region_name,

COUNT(pol.policy_id) AS policy_count,

**ROUND(SUM(CASE WHEN pay.payment_status = 'SUCCESS' THEN pay.paid_amount_inr
ELSE 0 END), 2) AS collected_premium_inr**

FROM regions r

LEFT JOIN app_users u ON u.region_id = r.region_id

LEFT JOIN quotes q ON q.sales_user_id = u.user_id

LEFT JOIN policies pol ON pol.quote_id = q.quote_id

LEFT JOIN payments pay ON pay.quote_id = q.quote_id

GROUP BY r.region_id, r.region_name

ORDER BY collected_premium_inr DESC;

425 -- Q11) Region-wise policy count and premium collection using sales user region.

426 SELECT

427 r.region_name,

428 COUNT(pol.policy_id) AS policy_count,

429 ROUND(SUM(CASE WHEN pay.payment_status = 'SUCCESS' THEN pay.paid_amount_inr ELSE 0 END), 2) AS collected_premium_inr

430 FROM regions r

431 LEFT JOIN app_users u ON u.region_id = r.region_id

432 LEFT JOIN quotes q ON q.sales_user_id = u.user_id

433 LEFT JOIN policies pol ON pol.quote_id = q.quote_id

434 LEFT JOIN payments pay ON pay.quote_id = q.quote_id

435 GROUP BY r.region_id, r.region_name

436 ORDER BY collected_premium_inr DESC;

region_name	policy_count	collected_premium_inr
West	5	60988.00
North	5	37406.00
South	0	0.00
East	0	0.00
Central	0	0.00
NorthEast	0	0.00
SouthWest	0	0.00
NorthWest	0	0.00
Coastal	0	0.00
Metro	0	0.00

Result 25 x

Output

#	Time	Action	Message
54	13.22.34	SELECT q.quote_no, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, v.registration_no, v.make, v.model, q.total_premi...	10 row(s) returned
55	13.24.10	SELECT b.broker_code, b.broker_org_name, COUNT(q.quote_id) AS quote_count FROM brokers b LEFT JOIN quotes q ON q.broker_id ...	10 row(s) returned
56	13.25.09	SELECT q.quote_no, q.quote_status, pay.payment_status, pol.policy_status FROM quotes q JOIN payments pay ON pay.quote_id = q...	3 row(s) returned
57	13.25.57	SELECT r.region_name, COUNT(pol.policy_id) AS policy_count, ROUND(SUM(CASE WHEN pay.payment_status = 'SUCCESS' THEN pa...	10 row(s) returned

Q12) Top 3 brokers by successful premium collection.

SELECT

b.broker_org_name,

ROUND(SUM(pay.paid_amount_inr), 2) AS collected_premium_inr

FROM brokers b

JOIN quotes q ON q.broker_id = b.broker_id

JOIN payments pay ON pay.quote_id = q.quote_id

WHERE pay.payment_status = 'SUCCESS'

GROUP BY b.broker_id, b.broker_org_name

ORDER BY collected_premium_inr DESC

LIMIT 3;

437

438 -- Q12) Top 3 brokers by successful premium collection.

439 • SELECT

440 b.broker_org_name,

441 ROUND(SUM(pay.paid_amount_inr), 2) AS collected_premium_inr

442 FROM brokers b

443 JOIN quotes q ON q.broker_id = b.broker_id

444 JOIN payments pay ON pay.quote_id = q.quote_id

445 WHERE pay.payment_status = 'SUCCESS'

446 GROUP BY b.broker_id, b.broker_org_name

447 ORDER BY collected_premium_inr DESC

448 LIMIT 3;

broker_org_name	collected_premium_inr
Apex West Insurance	21712.00
NorthEast Secure Brokers	20532.00
EastLine Risk Partners	18644.00

Result 26 x

Output

#	Time	Action	Message
55	13:24:10	SELECT	b.broker_code, b.broker_org_name, COUNT(q.quote_id) AS quote_count FROM brokers b LEFT JOIN quotes q ON q.broker_id ... 10 row(s) returned
56	13:25:09	SELECT	q.quote_no, q.quote_status, pay.payment_status, pol.policy_status FROM quotes q JOIN payments pay ON pay.quote_id = q.... 3 row(s) returned
57	13:25:57	SELECT	r.region_name, COUNT(pol.policy_id) AS policy_count, ROUND(SUM(CASE WHEN pay.payment_status = 'SUCCESS' THEN pa... 10 row(s) returned
58	13:26:33	SELECT	b.broker_org_name, ROUND(SUM(pay.paid_amount_inr), 2) AS collected_premium_inr FROM brokers b JOIN quotes q ON q.broke... 3 row(s) returned

Q13) Customers whose total quoted premium is above 20000.

SELECT

c.customer_id,

CONCAT(c.first_name, ' ', c.last_name) AS customer_name,

ROUND(SUM(q.total_premium), 2) AS total_quote_premium

FROM customers c

JOIN quotes q ON q.customer_id = c.customer_id

GROUP BY c.customer_id, c.first_name, c.last_name

HAVING SUM(q.total_premium) > 20000

ORDER BY total_quote_premium DESC;

```

449
450 -- Q13) Customers whose total quoted premium is above 20000.
451 • SELECT
452     c.customer_id,
453     CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
454     ROUND(SUM(q.total_premium), 2) AS total_quote_premium
455 FROM customers c
456 JOIN quotes q ON q.customer_id = c.customer_id
457 GROUP BY c.customer_id, c.first_name, c.last_name
458 HAVING SUM(q.total_premium) > 20000
459 ORDER BY total_quote_premium DESC;
460

```

customer_id	customer_name	total_quote_premium
5	Rohit Kapoor	33040.00
8	Pooja Singh	23010.00
9	Joseph Mathew	20532.00

Result 27 x

Output

#	Time	Action	Message
56	13.25.09	SELECT q.quote_no, q.quote_status, pay.payment_status, pol.policy_status FROM quotes q JOIN payments pay ON pay.quote_id = q...	3 row(s) returned
57	13.25.57	SELECT r.region_name, COUNT(pol.policy_id) AS policy_count, ROUND(SUM(CASE WHEN pay.payment_status = 'SUCCESS' THEN pa...	10 row(s) returned
58	13.26.33	SELECT b.broker_org_name, ROUND(SUM(paid_amount_inr), 2) AS collected_premium_inr FROM brokers b JOIN quotes q ON q.broke...	3 row(s) returned
59	13.26.52	SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, ROUND(SUM(q.total_premium), 2) AS total_quote_pr...	3 row(s) returned

Q14) Currency-wise payment summary (successful payments only).

SELECT

paid_currency,

COUNT(*) AS transaction_count,

ROUND(SUM(paid_amount), 2) AS total_paid_in_currency,

ROUND(SUM(paid_amount_inr), 2) AS total_paid_in_inr

FROM payments

WHERE payment_status = 'SUCCESS'

GROUP BY paid_currency

ORDER BY total_paid_in_inr DESC;

Limit to 1000 rows

```

461 -- Q14) Currency-wise payment summary (successful payments only).
462 SELECT
463     paid_currency,
464     COUNT(*) AS transaction_count,
465     ROUND(SUM(paid_amount), 2) AS total_paid_in_currency,
466     ROUND(SUM(paid_amount_inr), 2) AS total_paid_in_inr
467 FROM payments
468 WHERE payment_status = 'SUCCESS'
469 GROUP BY paid_currency
470 ORDER BY total_paid_in_inr DESC;

```

paid_currency	transaction_count	total_paid_in_currency	total_paid_in_inr
INR	5	67732.00	67732.00
AED	1	900.00	20532.00
USD	1	120.00	10030.00

Result Grid | Filter Rows: | Export | Wrap Cell Contents:

Result 29 x

Output

#	Time	Action	Message
58	13:26:33	SELECT	b.broker_org_name, ROUND(SUM(pay.paid_amount_inr), 2) AS collected_premium_inr FROM brokers b JOIN quotes q ON q.broke... 3 row(s) returned
59	13:26:52	SELECT	c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, ROUND(SUM(q.total_premium), 2) AS total_quote_pr... 3 row(s) returned
60	13:27:31	SELECT	paid_currency, COUNT(*) AS transaction_count, ROUND(SUM(paid_amount), 2) AS total_paid_in_currency, ROUND(SUM(pa... 3 row(s) returned
61	13:27:42	SELECT	paid_currency, COUNT(*) AS transaction_count, ROUND(SUM(paid_amount), 2) AS total_paid_in_currency, ROUND(SUM(pa... 3 row(s) returned

Read Only

Q15) Commission payable per broker from successful payments.

SELECT

b.broker_code,

b.broker_org_name,

b.commission_pct,

ROUND(SUM(pay.paid_amount_inr), 2) AS successful_premium,

**ROUND(SUM(pay.paid_amount_inr) * b.commission_pct / 100, 2) AS
commission_payable**

FROM brokers b

JOIN quotes q ON q.broker_id = b.broker_id

JOIN payments pay ON pay.quote_id = q.quote_id

WHERE pay.payment_status = 'SUCCESS'

GROUP BY b.broker_id, b.broker_code, b.broker_org_name, b.commission_pct

ORDER BY commission_payable DESC;

```

471
472 -- Q15) Commission payable per broker from successful payments.
473 * SELECT
474     b.broker_code,
475     b.broker_org_name,
476     b.commission_pct,
477     ROUND(SUM(pay.paid_amount_inr), 2) AS successful_premium,
478     ROUND(SUM(pay.paid_amount_inr) * b.commission_pct / 100, 2) AS commission_payable
479 FROM brokers b
480 JOIN quotes q ON q.broker_id = b.broker_id
481 JOIN payments pay ON pay.quote_id = q.quote_id
482 WHERE pay.payment_status = 'SUCCESS'
483 GROUP BY b.broker_id, b.broker_code, b.broker_org_name, b.commission_pct
484 ORDER BY commission_payable DESC;
485

```

broker_code	broker_org_name	commission_pct	successful_premium	commission_payable
BRK-W001	Apex West Insurance	11.00	21712.00	2388.32
BRK-NE01	NorthEast Secure Brokers	9.50	20532.00	1950.54
BRK-ED01	EastLine Risk Partners	9.75	18644.00	1817.79
BRK-N001	Monarch North Brokers	12.50	14160.00	1770.00
BRK-C001	Central Cover House	10.25	13216.00	1354.64

Result 30 x

Output

#	Time	Action	Message
59	13:26:52	SELECT c.customer_id, CONCAT(c.first_name, ' ' c.last_name) AS customer_name, ROUND(SUM(q.total_premium), 2) AS total_quote_gr...	3 row(s) returned
60	13:27:31	SELECT paid_currency, COUNT(*) AS transaction_count, ROUND(SUM(paid_amount), 2) AS total_paid_in_currency, ROUND(SUM(pa...	3 row(s) returned
61	13:27:42	SELECT paid_currency, COUNT(*) AS transaction_count, ROUND(SUM(paid_amount), 2) AS total_paid_in_currency, ROUND(SUM(pa...	3 row(s) returned
62	13:28:34	SELECT b.broker_code, b.broker_org_name, b.commission_pct, ROUND(SUM(pay.paid_amount_inr), 2) AS successful_premium, R...	5 row(s) returned

Window Functions

Q16) LAG and LEAD for premium trend across quote timeline.

SELECT

q.quote_no,

q.created_at,

q.total_premium,

LAG(q.total_premium) OVER (
ORDER BY q.created_at, q.quote_id

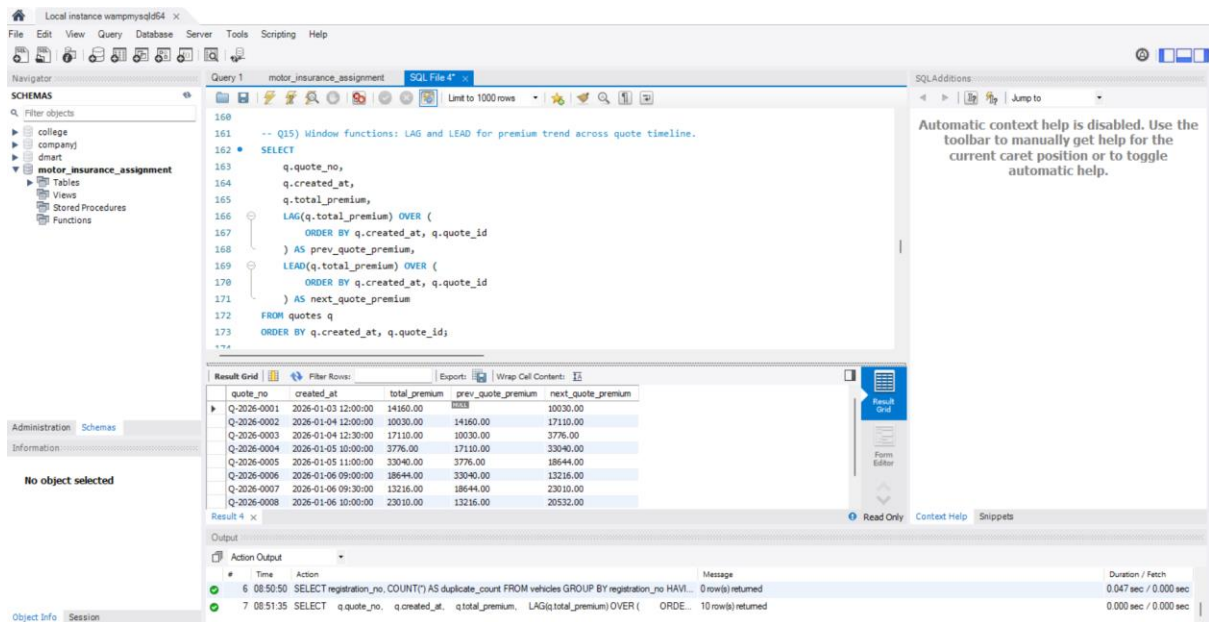
) AS prev_quote_premium,

LEAD(q.total_premium) OVER (
ORDER BY q.created_at, q.quote_id

) AS next_quote_premium

FROM quotes q

ORDER BY q.created_at, q.quote_id;



Q17) ROW_NUMBER, RANK, DENSE_RANK by region.

SELECT

r.region_name,

q.quote_no,

q.total_premium,

ROW_NUMBER() OVER (

PARTITION BY r.region_name

ORDER BY q.total_premium DESC

) AS row_num_in_region,

RANK() OVER (

PARTITION BY r.region_name

ORDER BY q.total_premium DESC

) AS rank_in_region,

DENSE_RANK() OVER (

PARTITION BY r.region_name

ORDER BY q.total_premium DESC

) AS dense_rank_in_region

FROM quotes q

JOIN app_users u ON u.user_id = q.sales_user_id

JOIN regions r ON r.region_id = u.region_id

ORDER BY r.region_name, row_num_in_region;

The screenshot shows a MySQL IDE window titled 'Local instance wampmysqld64'. The 'Query 1' tab is active, displaying a SQL query for window functions. The query is as follows:

```
-- Q16) Window functions: ROW_NUMBER, RANK, DENSE_RANK by region.
139
140 SELECT
141     r.region_name,
142     q.quote_no,
143     q.total_premium,
144     ROW_NUMBER() OVER (
145         PARTITION BY r.region_name
146         ORDER BY q.total_premium DESC
147     ) AS row_num_in_region,
148     RANK() OVER (
149         PARTITION BY r.region_name
150         ORDER BY q.total_premium DESC
151     ) AS rank_in_region,
152     DENSE_RANK() OVER (
153         PARTITION BY r.region_name
154         ORDER BY q.total_premium DESC
155     ) AS dense_rank_in_region
156 FROM regions r
157 JOIN quotes q ON r.region_id = q.region_id
158 ORDER BY r.region_name, row_num_in_region;
```

The 'Result Grid' shows the following data:

region_name	quote_no	total_premium	row_num_in_region	rank_in_region	dense_rank_in_region
North	Q-2026-0008	23010.00	1	1	1
North	Q-2026-0001	14160.00	2	2	2
North	Q-2026-0007	13216.00	3	3	3
North	Q-2026-0002	10030.00	4	4	4
North	Q-2026-0004	3776.00	5	5	5
West	Q-2026-0005	33040.00	1	1	1
West	Q-2026-0009	20532.00	2	2	2
West	Q-2026-0006	18644.00	3	3	3

The 'Output' tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
7	08:51:35	SELECT q.quote_no, q.created_at, q.total_premium, LAG(q.total_premium) OVER (	ORDE... 10 row(s) returned	0.000 sec / 0.000 sec
8	08:56:48	SELECT r.region_name, q.quote_no, q.total_premium, ROW_NUMBER() OVER (	PARTITL... 10 row(s) returned	0.000 sec / 0.000 sec

Q18) Indexing and optimization check with explain keyword

CREATE INDEX idx_quotes_status_created ON quotes (quote_status, created_at);

EXPLAIN

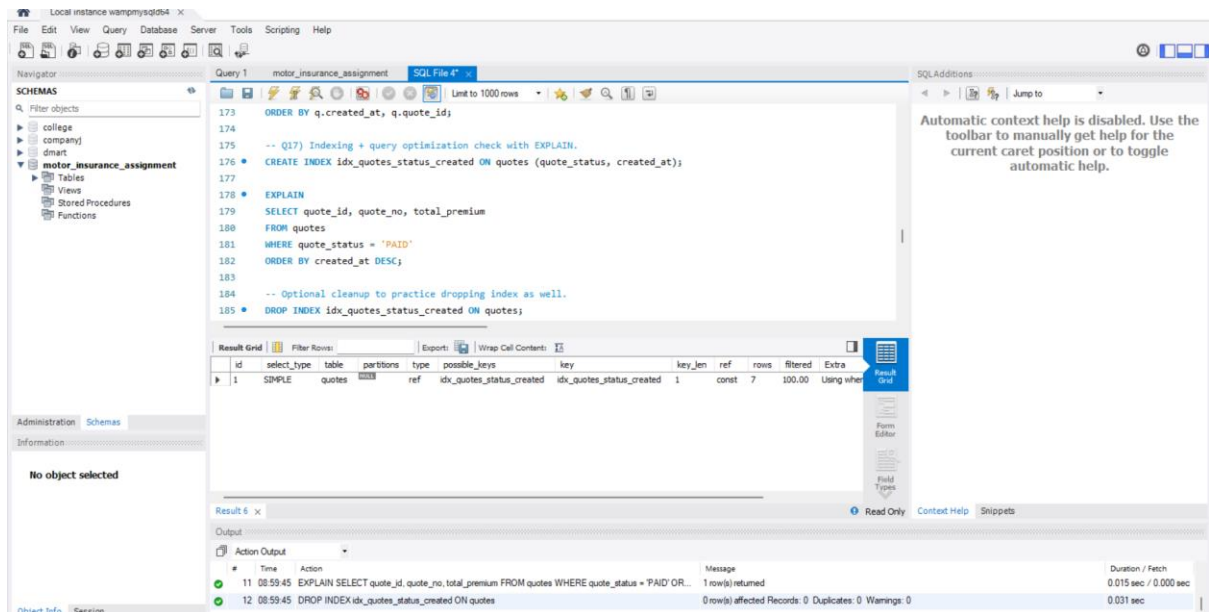
SELECT quote_id, quote_no, total_premium

FROM quotes

WHERE quote_status = 'PAID'

ORDER BY created_at DESC;

DROP INDEX idx_quotes_status_created ON quotes;



Q19) Transactions: START TRANSACTION, SAVEPOINT, ROLLBACK, COMMIT.

START TRANSACTION;

UPDATE brokers

SET prepaid_balance = prepaid_balance - 1000

WHERE broker_id = 1;

SAVEPOINT sp_after_first_update;

UPDATE brokers

SET prepaid_balance = prepaid_balance - 500

WHERE broker_id = 2;

ROLLBACK TO sp_after_first_update;

UPDATE brokers

SET prepaid_balance = prepaid_balance + 1000

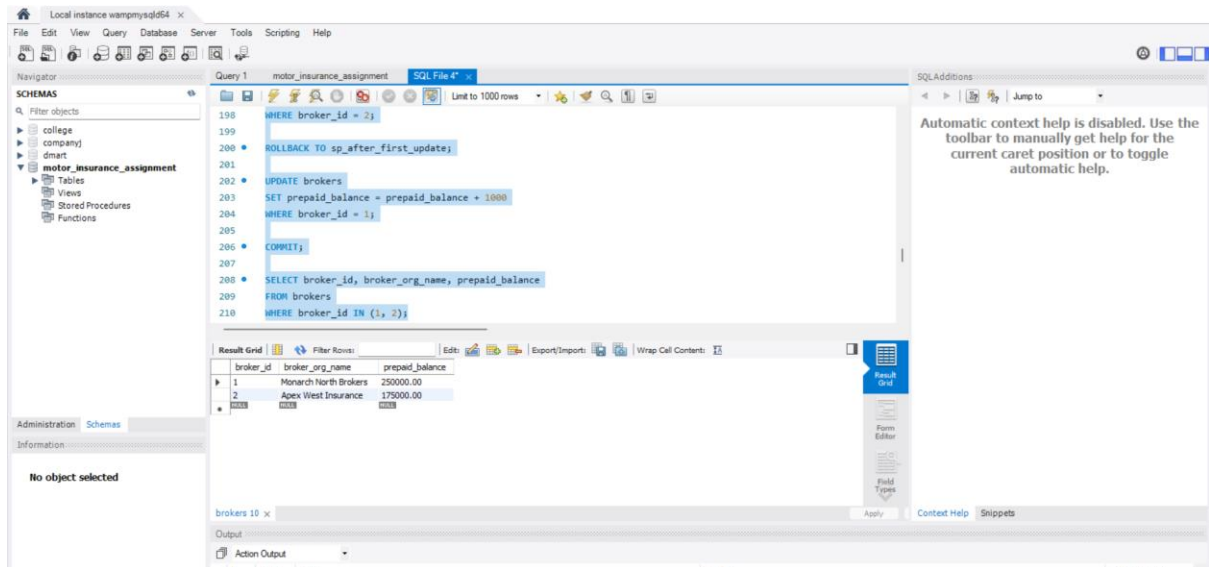
WHERE broker_id = 1;

COMMIT;

SELECT broker_id, broker_org_name, prepaid_balance

FROM brokers

WHERE broker_id IN (1, 2);



Q20) Views

CREATE OR REPLACE VIEW vw_region_policy_collection AS

SELECT

r.region_id,

r.region_name,

COUNT(DISTINCT pol.policy_id) AS policy_count,

**ROUND(SUM(CASE WHEN pay.payment_status = 'SUCCESS' THEN pay.paid_amount_inr
ELSE 0 END), 2) AS collected_premium_inr**

FROM regions r

LEFT JOIN app_users u ON u.region_id = r.region_id

LEFT JOIN quotes q ON q.sales_user_id = u.user_id

LEFT JOIN policies pol ON pol.quote_id = q.quote_id

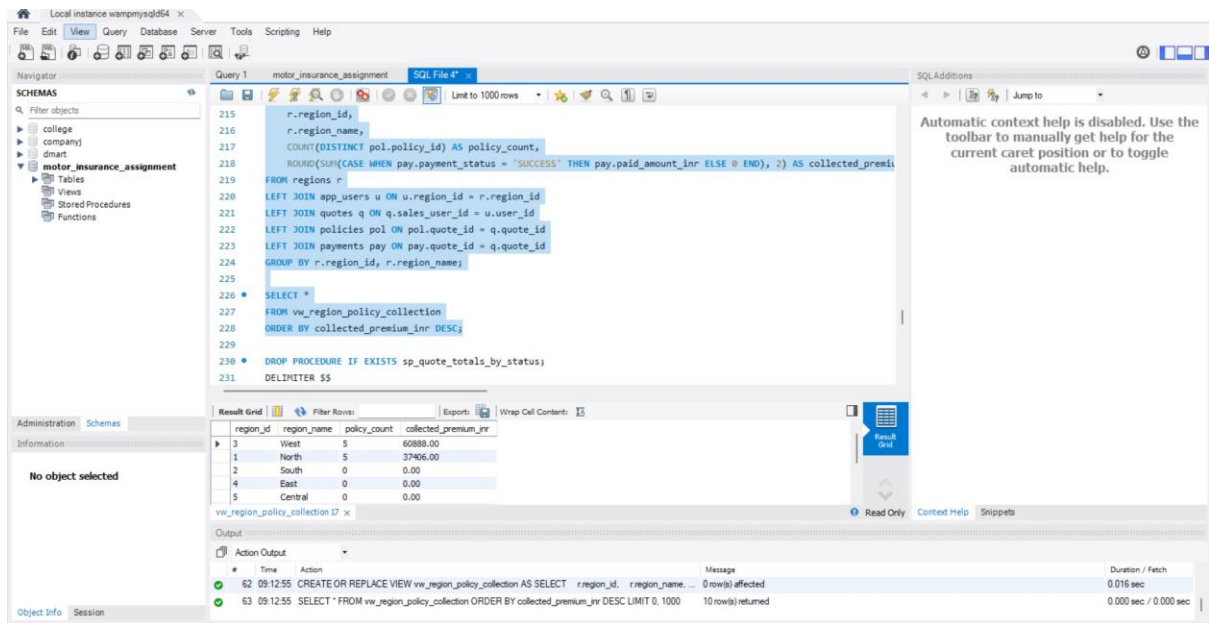
LEFT JOIN payments pay ON pay.quote_id = q.quote_id

GROUP BY r.region_id, r.region_name;

SELECT *

FROM vw_region_policy_collection

ORDER BY collected_premium_inr DESC;



Q21) RIGHT JOIN: Show all quotes and broker details when available.

SELECT

q.quote_no,

q.source_channel,

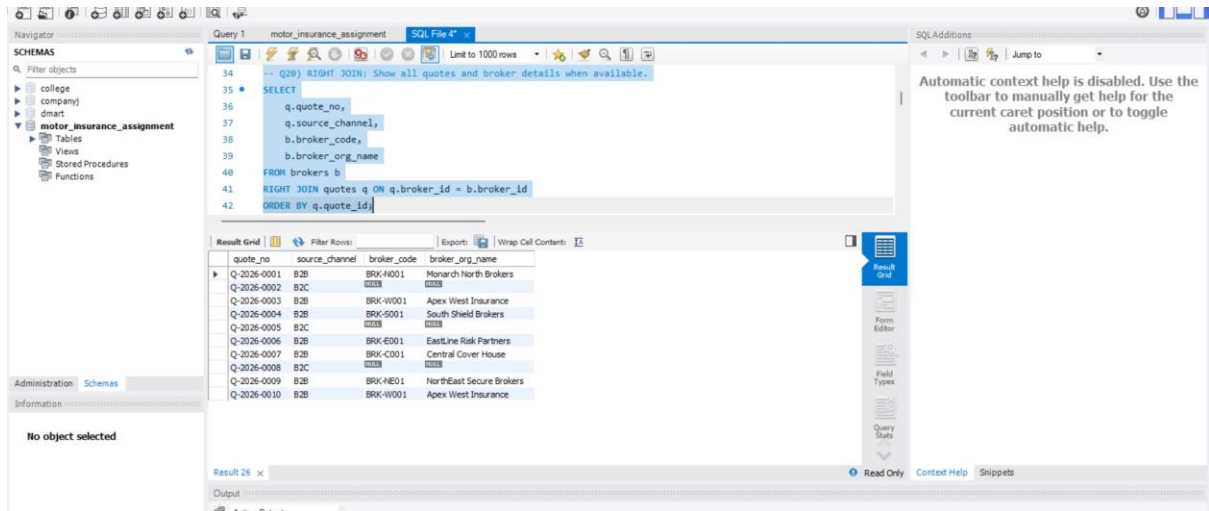
b.broker_code,

b.broker_org_name

FROM brokers b

RIGHT JOIN quotes q ON q.broker_id = b.broker_id

ORDER BY q.quote_id;



Q22) FULL OUTER JOIN (LEFT + RIGHT + UNION ALL).

SELECT

b.broker_code,

b.broker_org_name,

q.quote_no,

q.quote_status

FROM brokers b

LEFT JOIN quotes q ON q.broker_id = b.broker_id

UNION ALL

SELECT

b.broker_code,

b.broker_org_name,

q.quote_no,

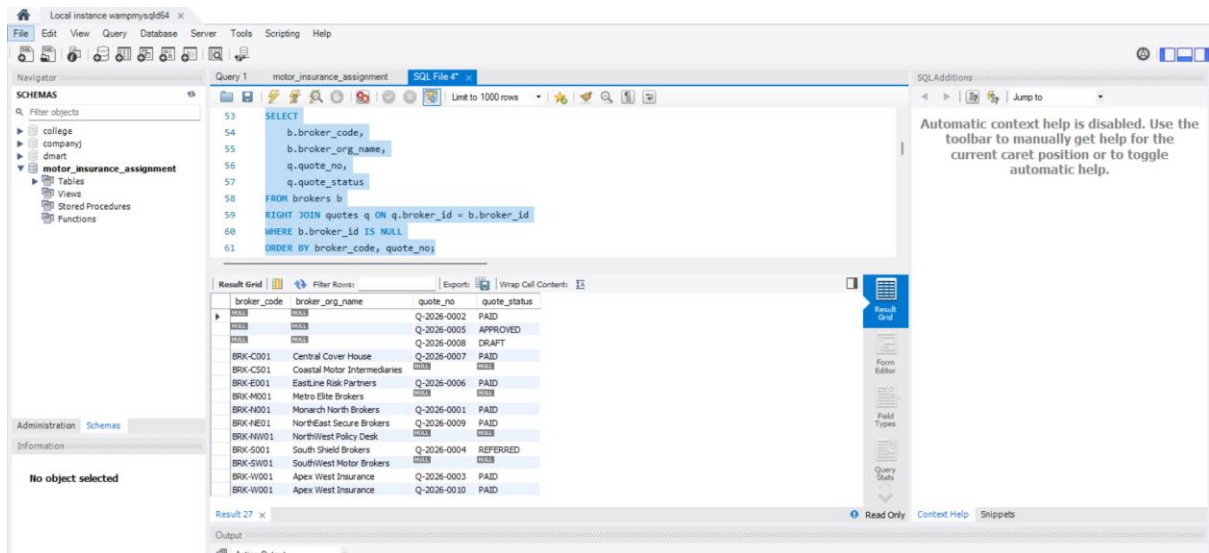
q.quote_status

FROM brokers b

RIGHT JOIN quotes q ON q.broker_id = b.broker_id

WHERE b.broker_id IS NULL

ORDER BY broker_code, quote_no;



Q23) Subquery in WHERE: Customers who have at least one high-value quote.

SELECT

c.customer_id,

CONCAT(c.first_name, ' ', c.last_name) AS customer_name

FROM customers c

WHERE c.customer_id IN (

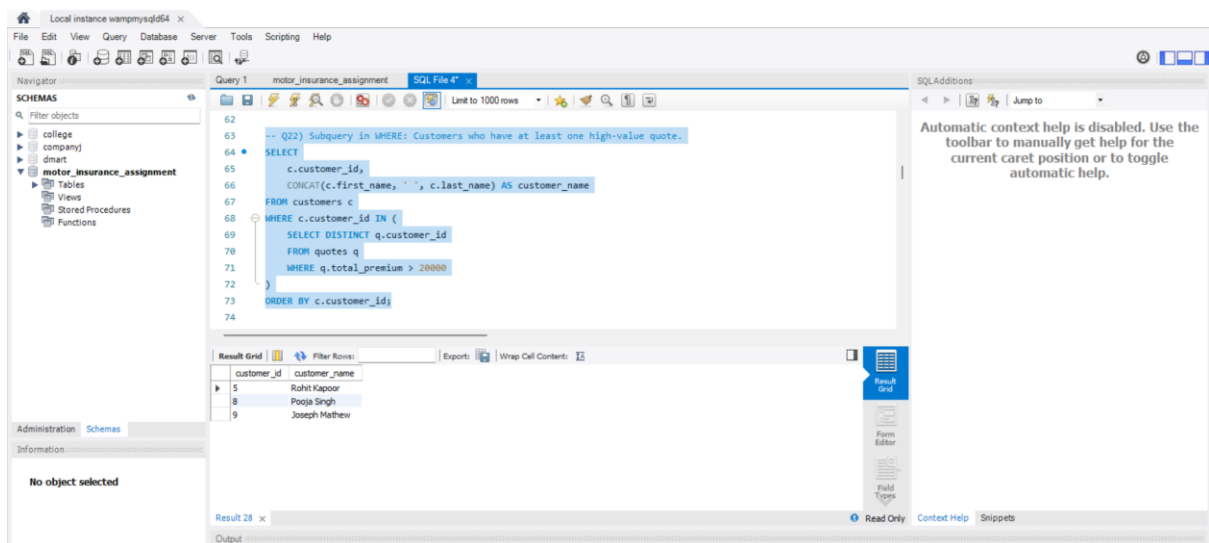
SELECT DISTINCT q.customer_id

FROM quotes q

WHERE q.total_premium > 20000

)

ORDER BY c.customer_id;



Q24) Subquery in FROM (derived table): Channels with avg premium above 15000.

SELECT

dt.source_channel,

ROUND(dt.avg_premium, 2) AS avg_premium

FROM (

SELECT source_channel, AVG(total_premium) AS avg_premium

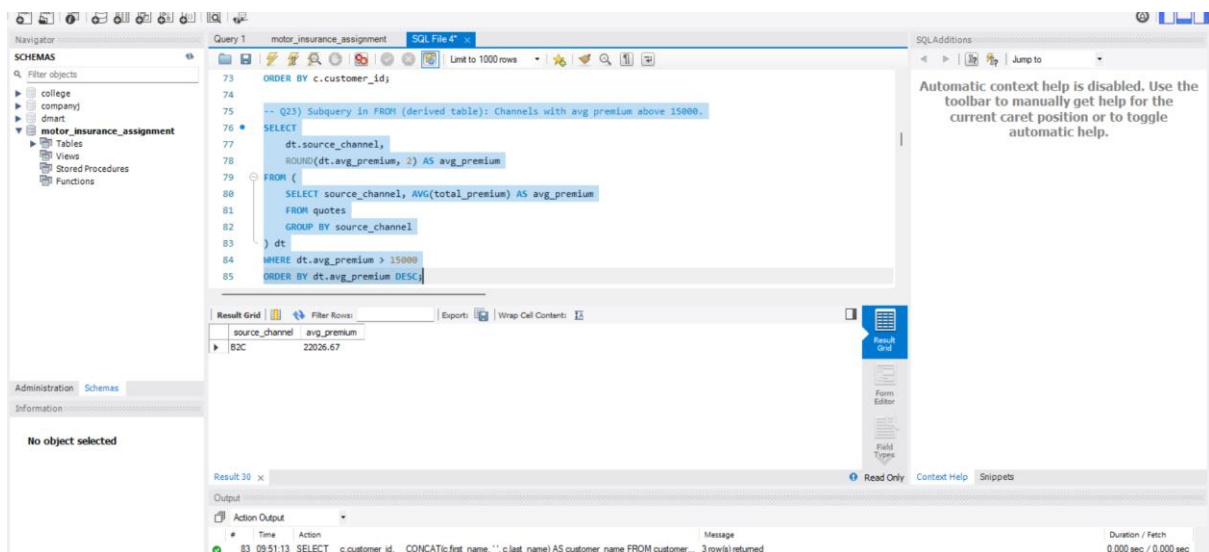
FROM quotes

GROUP BY source_channel

) dt

WHERE dt.avg_premium > 15000

ORDER BY dt.avg_premium DESC;



Q25) Subquery with ALL operator

Quotes higher than ALL TPL quote premiums.

SELECT

q.quote_no,

q.total_premium

FROM quotes q

```

WHERE q.total_premium > ALL (

    SELECT q2.total_premium

    FROM quotes q2

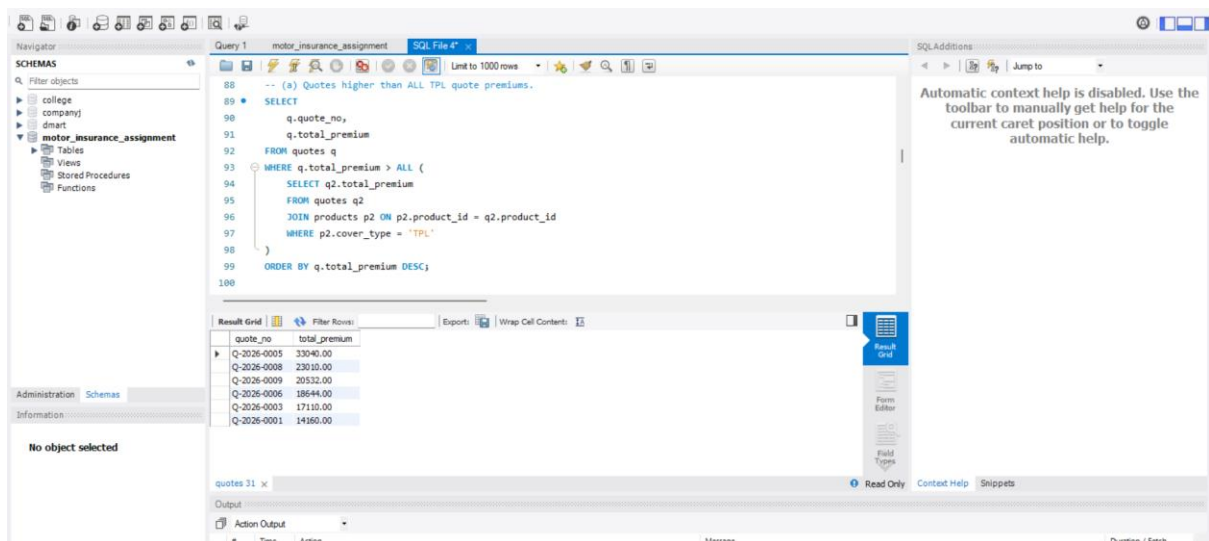
    JOIN products p2 ON p2.product_id = q2.product_id

    WHERE p2.cover_type = 'TPL'

)

ORDER BY q.total_premium DESC;

```



Q26) Subquery with ANY operator

```

SELECT

    q.quote_no,

    q.total_premium

FROM quotes q

WHERE q.total_premium >= ANY (

    SELECT q2.total_premium

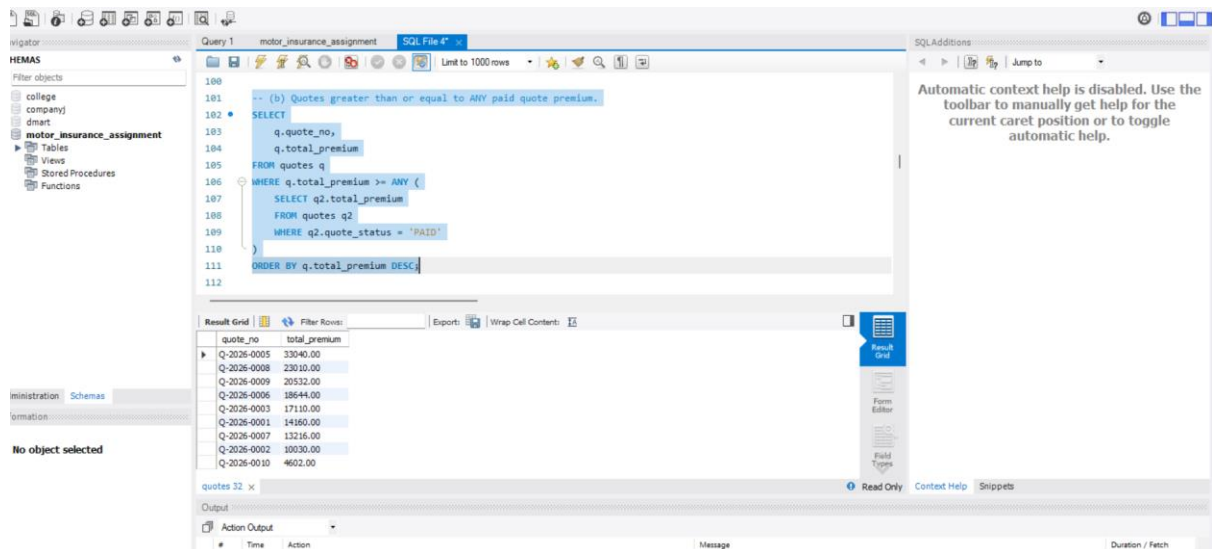
    FROM quotes q2

    WHERE q2.quote_status = 'PAID'

)

ORDER BY q.total_premium DESC;

```



Q27) Advanced SQL: CASE + BETWEEN + IN + LIKE + LIMIT + OFFSET.

SELECT

q.quote_no,

q.total_premium,

CASE

WHEN q.total_premium >= 25000 THEN 'HIGH_VALUE'

WHEN q.total_premium BETWEEN 10000 AND 24999.99 THEN 'MEDIUM_VALUE'

ELSE 'LOW_VALUE'

END AS premium_band

FROM quotes q

JOIN customers c ON c.customer_id = q.customer_id

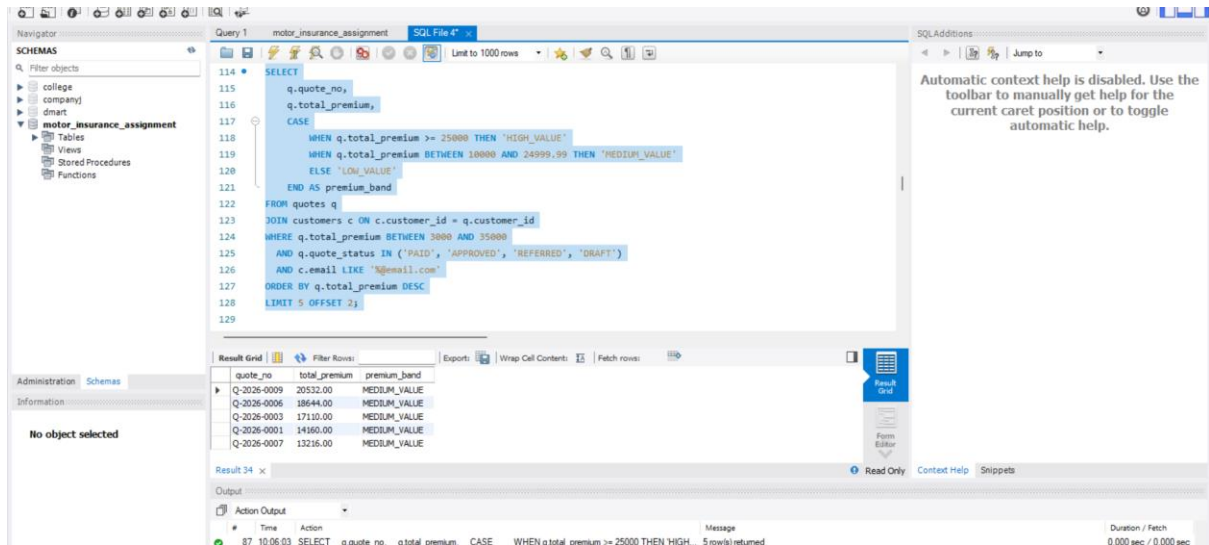
WHERE q.total_premium BETWEEN 3000 AND 35000

AND q.quote_status IN ('PAID', 'APPROVED', 'REFERRED', 'DRAFT')

AND c.email LIKE '%@email.com'

ORDER BY q.total_premium DESC

LIMIT 5 OFFSET 2;



Q28) Create a stored procedure with INOUT parameter to increase premium by 5%.

DROP PROCEDURE IF EXISTS sp_increase_premium_5pct;

DELIMITER \$\$

CREATE PROCEDURE sp_increase_premium_5pct(INOUT p_amount DECIMAL(12,2))

BEGIN

DECLARE v_increment DECIMAL(12,2);

IF p_amount <= 0 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Premium amount must be greater than zero';

END IF;

SET v_increment = ROUND(p_amount * 0.05, 2);

SET p_amount = ROUND(p_amount + v_increment, 2);

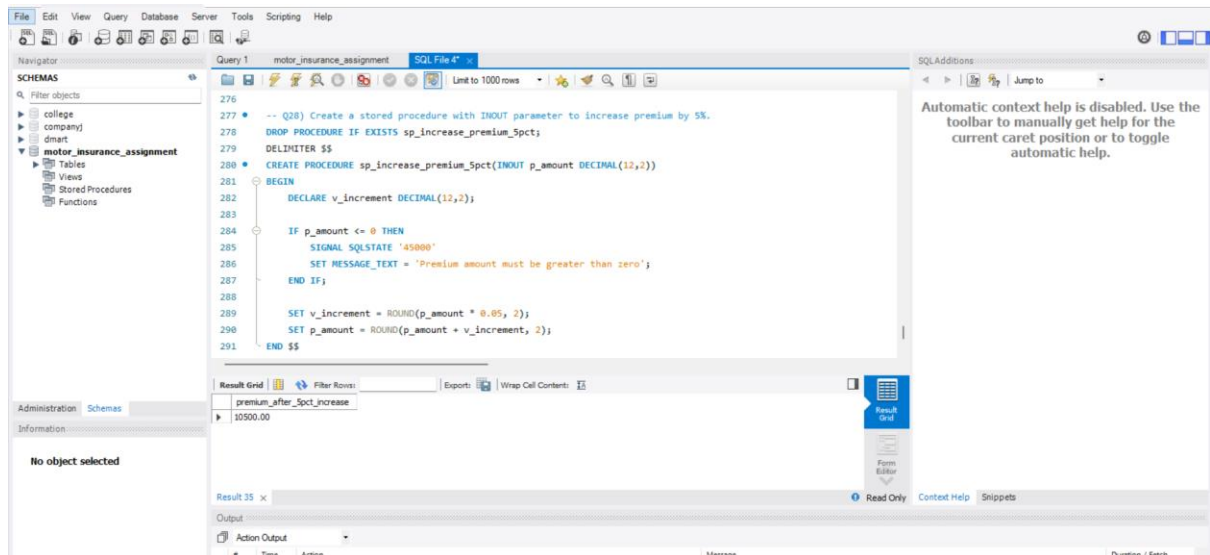
END \$\$

DELIMITER ;

SET @premium_amount = 10000.00;

CALL sp_increase_premium_5pct(@premium_amount);

SELECT @premium_amount AS premium_after_5pct_increase;



Q29) Create trigger to block duplicate vehicle registration number.

DROP TRIGGER IF EXISTS trg_vehicles_before_insert_no_duplicate_reg;

DELIMITER \$\$

CREATE TRIGGER trg_vehicles_before_insert_no_duplicate_reg

BEFORE INSERT ON vehicles

FOR EACH ROW

BEGIN

IF EXISTS (

SELECT 1

FROM vehicles v

WHERE v.registration_no = NEW.registration_no

) THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Duplicate registration number is not allowed';

END IF;

END \$\$

DELIMITER ;

Q30) Divide quotes into 4 premium groups using NTILE(4).

SELECT

q.quote_no,

q.total_premium,

NTILE(4) OVER (ORDER BY q.total_premium DESC) AS premium_quartile

FROM quotes q

ORDER BY q.total_premium DESC;

The screenshot shows a SQL IDE interface with a query editor, a result grid, and a schema navigator. The query editor contains the following SQL code:

```
318
319
320
321
322
323
324
325 -- Q10) Divide quotes into 4 premium groups using NTILE(4).
326 SELECT
327     q.quote_no,
328     q.total_premium,
329     NTILE(4) OVER (ORDER BY q.total_premium DESC) AS premium_quartile
330 FROM quotes q
331 ORDER BY q.total_premium DESC;
332
333
```

The result grid displays the following data:

quote_no	total_premium	premium_quartile
Q-2026-0005	33040.00	1
Q-2026-0008	23010.00	1
Q-2026-0009	20532.00	1
Q-2026-0006	18644.00	2
Q-2026-0003	17110.00	2
Q-2026-0001	14160.00	2

The schema navigator on the left shows the following structure:

- Filter objects
- college
- companyj
- dmart
- motor_insurance_assignment
 - Tables
 - Views
 - Stored Procedures
 - Functions

The output pane at the bottom shows the following message:

```
97 10:25:39 CREATE TRIGGER trg_vehicles_before_insert_no_duplicate_reg BEFORE INSERT ON vehicles FOR E...
0 row(s) affected
Duration / Fetch
0.016 sec
```

